

Design and Implementation of AgriXplain: An IoT-Enabled Open Smart Farming Platform with Multimodal Sensing, Explainable Ensemble Learning, and Full-Stack Cloud Analytics

Kshitija Sole, Author Two, Author Three, Dr. Anil Singh

Kshitija Sole is with the Department of Computer Science, Institution Name, City, Country (e-mail: email@institution.edu). Manuscript received Month Day, 2025; revised Month Day, 2025. This work was supported in part by [Funding Agency] under Grant DST/JSPS/INSPIRE/2024-00123. (Corresponding author: Kshitija Sole.)

Abstract—Open-source precision agriculture platforms occupy an unusual position in the research-to-practice pipeline. They are plentiful enough to fill a survey paper, yet rare enough in genuine end-to-end completeness—hardware wiring, server inference, database persistence, web frontend, mobile packaging, and explanation generation all working together—that practitioners seeking a deployable reference implementation consistently report starting from scratch. This paper describes AgriXplain, our attempt to address that gap with a single, fully documented, self-hostable stack. The edge station is an ESP32-WROOM-32 acquiring ten sensor modalities: air temperature and humidity via DHT22, volumetric soil moisture and pH proxy via analog front-ends on ADC1 channels, light intensity via LDR voltage divider, binary rainfall state, water flow via hall-effect turbine meter with ISR pulse counting, wind proxy via pulse anemometer, and soil NPK via Modbus RTU holding registers. Telemetry is transmitted as structured JSON over Wi-Fi to a FastAPI backend running four inference models per reading: Random Forest, XGBoost, Bayesian Ridge, and AgriFusion—a proposed weighted softmax ensemble that blends the RF and XGBoost class probability vectors with fixed weights (0.35 / 0.65) and applies a disagreement-aware confidence damping (factor 0.88, floor 0.35) when the two base learners predict different classes. TreeSHAP attributions are computed and persisted with every sensor reading, providing crop-specific intervention explanations surfaced in the farmer dashboard. On a reproducible stratified hold-out evaluation from the synthetic benchmark dataset shipped with the platform, AgriFusion achieves 70.68% accuracy and 69.97% weighted F1 score, exceeding individual base learner performance. The platform integrates role-differentiated farmer and admin dashboards, multi-tenant farm management, data export, and a thermal-colormap vision pipeline with heuristic plant health diagnosis. We provide GPIO-level hardware documentation, firmware pseudocode, the full REST API surface, relational schema semantics, a reproducible training pipeline, and a structured comparison against typical proprietary agri-IoT bundles on calibration transparency, analytics depth, data ownership, and extensibility. Limitations of synthetic training data, heuristic vision, and simplified sensor calibrations are stated explicitly, alongside a multi-season field trial agenda required for validated agronomic deployment.

Index Terms—Smart agriculture, Internet of Things, ESP32, Modbus RTU, soil sensing, cloud computing, machine learning, XGBoost, Random Forest, ensemble learning, explainable artificial intelligence, SHAP, AgriFusion, FastAPI, Next.js, Capacitor, computer vision, plant disease, sensor fusion, precision farming, data sovereignty

I. INTRODUCTION

Agricultural productivity faces compound pressures from a projected 50 percent increase in global food demand by 2050, declining freshwater availability, and accelerating climate variability [1]. Precision

agriculture—the practice of applying spatially and temporally differentiated inputs informed by field data—offers a pathway to sustaining yields while reducing agrochemical and water inputs [3],[4]. The infrastructure prerequisite for this approach—wireless soil sensors, edge microcontrollers, and cloud-hosted

inference—is now accessible at costs well within reach of smallholder cooperatives and research extension programs. A five-dollar ESP32 module, a thirty-dollar NPK probe, and a cloud server costing under twenty dollars per month are sufficient hardware for a functional precision farming node. The barrier, as of writing, is not hardware cost but software depth and auditability.

Two persistent gaps limit adoption by research cooperatives and regulatory bodies. The openness gap: most deployable smart-farming systems are commercial bundles whose calibration polynomials, model weights, and training pipelines are inaccessible to operators, precluding independent validation, third-party integration, or site-specific calibration refinement [16],[36]. We reviewed seven commercial precision agriculture platforms during our preparatory work; none published their calibration methodology; five required a subscription cloud service with no data export API; all seven returned a single opaque risk score with no explanation of the contributing sensor features. The transparency gap: even open-source prototypes rarely provide per-reading machine learning explanations, multi-model comparison, or documented schemas that support audit trails [26],[28]. Both gaps are particularly consequential for irrigation and pesticide dosing decisions, where errors carry direct economic and environmental costs [13].

AgriXplain addresses both gaps through a four-component vertical stack. First, a modular ESP32 edge station with explicit calibration constants in firmware headers, interrupt-driven pulse metrology for flow and wind, and Modbus RTU NPK sensing—ten measurement channels transmitting as a single structured JSON payload. Second, a FastAPI backend with SQLAlchemy multi-tenant schema, six-stage inference pipeline, and 37 documented REST endpoints. Third, AgriFusion—a weighted softmax ensemble combining Random Forest and XGBoost class probability vectors with disagreement-aware confidence damping, with per-reading TreeSHAP attributions persisted in the relational database. Fourth, a Next.js 14 web application with Capacitor 6 Android packaging, role-differentiated farmer and admin dashboards, and heuristic computer vision plant health analysis.

The positioning of this work relative to the existing literature is deliberate. AgriXplain is not primarily a

new ML algorithm paper—the ensemble design is a reasoned engineering choice, not a theoretical contribution. It is not a hardware paper—the ESP32 sensor suite is a carefully selected combination of commodity modules, not novel sensing elements. It is a systems paper: the contribution is the complete, documented, reproducible integration of hardware, firmware, backend, ML, explainability, and frontend into a single stack that any researcher or cooperative can deploy, inspect, and modify. The benchmark results validate correct operation; field trials—which we outline but have not yet completed—will validate agronomic utility.

A. Research Contributions

The specific contributions of this work are: (C1) a complete, file-level reproducible implementation of a multimodal smart farming platform whose every component—calibration constants, ML weights, database schema, API routes—is documented and modifiable without vendor access; (C2) the AgriFusion ensemble algorithm with formal description, pseudocode, disagreement-aware confidence mechanism, and synthetic benchmark evaluation showing consistent improvement over individual base learners; (C3) a systematic multi-criterion comparison against typical commercial agri-IoT bundles; and (C4) a structured multi-season field trial agenda and explicit limitation analysis.

B. Paper Organization

Section II provides an extended review across IoT agriculture, machine learning, explainable AI, and computer vision. Section III presents system design methodology. Sections IV–V detail hardware and firmware. Section VI covers the backend. Section VII presents ML methodology including AgriFusion and evaluation. Section VIII covers computer vision. Section IX describes the web and mobile tiers. Section X addresses security. Sections XI–XV cover evaluation, comparison, engineering decisions, calibration, and extended analysis. Section XVI concludes. Appendices provide GPIO maps, API catalog, schema documentation, ML pseudocode, and glossary.

A. Research Contributions

The specific contributions of this work are: (C1) a complete, file-level reproducible implementation of a multimodal smart farming platform whose every

component—calibration constants, ML weights, database schema, API routes—is documented and modifiable; (C2) the AgriFusion ensemble algorithm with formal description, pseudocode, and benchmark evaluation showing improvement over individual base learners; (C3) a systematic comparison of the platform against typical commercial agri-IoT bundles on transparency, data ownership, analytics depth, and extensibility dimensions; (C4) a structured field trial agenda and limitation analysis grounding the platform's agronomic validity claims.

B. Paper Organization

Section II provides an extended review of related work across IoT agriculture, machine learning, explainable AI, and computer vision. Section III presents the system design methodology and requirements. Section IV details the hardware architecture and sensor characterization. Section V covers firmware design with measurement equations. Section VI presents backend architecture, data models, and API surface. Section VII details the ML methodology including AgriFusion formulation and evaluation. Section VIII covers computer vision. Section IX describes the web and mobile tiers. Section X addresses security and deployment. Section XI presents evaluation including field trial protocol. Section XII compares with commercial solutions. Section XIII presents a discussion of findings and limitations. Section XIV concludes with future work directions. Appendices provide complete GPIO maps, API catalog, schema documentation, ML pseudocode, and glossary.

II. RELATED WORK

A. IoT Architectures in Precision Agriculture

The IoT literature distinguishes perception, connectivity, processing, and application layers as the four tiers of a domain deployment [1],[2]. In agricultural contexts, the perception layer must accommodate heterogeneous physical modalities—electrochemical soil probes, optical leaf sensors, capacitive moisture probes, MEMS weather stations—with power, cost, and maintenance constraints not found in industrial IoT [20]. Atzori et al. [1] surveyed the early IoT landscape and identified the absence of standard service composition protocols; Gubbi et al. [2] proposed a cloud-centric architecture that presaged current FastAPI/cloud deployments. Elijah et al. [20] conducted an extensive review of IoT and data

analytics in agriculture, identifying crop monitoring, irrigation scheduling, and disease early warning as the highest value-of-information use cases, directly motivating AgriXplain's UC-2 through UC-6.

Tzounis et al. [21] surveyed practical IoT agriculture deployments and identified field connectivity (cellular vs. LoRa vs. Wi-Fi), data sovereignty, and interoperability as recurring challenges. AgriXplain targets Wi-Fi LAN deployments for infrastructure-rich farms and extensible to cellular via reverse proxy. Gondchawar and Kawitkar [22] presented one of the early ESP8266-based smart agriculture prototypes limited to temperature, humidity, and soil moisture; AgriXplain extends this to ten modalities with NPK and computer vision. Kassim [23] analyzed issues in IoT smart agriculture deployment including sensor calibration drift, network unreliability, and data security—all addressed in AgriXplain's firmware error handling, reconnect logic, and JWT/bcrypt security design. Farooq et al. [3] systematic review of 2013–2019 publications confirmed that end-to-end reproducible implementations remain scarce; Ahmad et al. [38] confirmed this gap persists in the AI-IoT convergence literature through 2021.

B. Machine Learning for Agricultural Classification

Liakos et al. [10] identified Random Forest as the most reported classifier across four precision agriculture application domains; their review also noted that feature engineering from heterogeneous sensors often dominates model accuracy improvements. Benos et al. [17] provided an updated 2021 review confirming that gradient boosting (XGBoost, LightGBM) has supplanted SVM as the second most common method for tabular sensor data. Chlingaryan et al. [18] specifically analyzed yield prediction and nitrogen estimation, finding that ensemble methods outperform single classifiers when training data covers multiple crop varieties—a property AgriXplain's synthetic generator approximates across seven crops. Sharma et al. [19] reviewed precision agriculture ML applications and noted the absence of deployment-ready APIs and multi-user support in academic implementations.

The theoretical basis for ensemble superiority is well-established. Dietterich [32] proved that ensembles of diverse, competent classifiers outperform individual

members when (a) base classifiers are accurate and (b) their errors are uncorrelated—conditions that Random Forest and XGBoost partially satisfy through bootstrap sampling and sequential residual fitting respectively. Polikar [33] surveyed AdaBoost, Bagging, and stacking variants; AgriFusion's weighted average is closest to stacking without a meta-learner, retaining interpretability of fixed weights. Wolfert et al. [16] identified big data integration as a prerequisite for ensemble farming at scale, motivating AgriXplain's multi-tenant design and export functionality. Pivoto et al. [25] traced the gap between smart farming prototypes and deployable systems, noting lack of UI and API as primary barriers; AgriXplain addresses both.

C. Explainable Artificial Intelligence in Agriculture

The XAI literature distinguishes ante-hoc interpretable models (decision trees, linear regression) from post-hoc explanation methods (LIME [30], SHAP [5], integrated gradients). Adadi and Berrada [26] surveyed the XAI landscape and categorized methods by model transparency, explanation scope (local vs. global), and output type (feature importance, rules, examples). Doshi-Velez and Kim [27] argued that interpretability evaluation must be grounded in specific human tasks; for AgriXplain's farmer use case, the relevant task is deciding whether to irrigate or apply fertilizer, making per-reading feature attribution the highest-value explanation format. Rudin [28] argued for ante-hoc interpretability for high-stakes decisions; AgriXplain's tree base learners are inherently interpretable, but SHAP provides a more granular and consistent attribution than raw tree paths.

Lundberg and Lee [5] unified additive feature attribution under the SHAP framework and proved that TreeSHAP satisfies local accuracy, consistency, and missingness axioms—properties that post-hoc linear surrogates (LIME) do not guarantee. Ribeiro et al. [30] proposed LIME as a model-agnostic alternative; for tree ensembles TreeSHAP is preferred due to exact computation rather than sampling approximation. Samek et al. [29] reviewed visualization methods for neural network explanations; for tabular AgriXplain sensor data a SHAP bar chart is more interpretable than saliency maps and directly actionable by agronomists. Di Vaio et al. [36] argued that AI transparency is a prerequisite

for sustainable agri-food supply chain governance, strengthening the case for AgriXplain's SHAP persistence and admin SHAP analytics dashboard.

D. Computer Vision for Plant Health Monitoring

Kamilaris and Prenafeta-Boldú [11] surveyed 40 deep learning applications in agriculture, reporting that convolutional neural networks achieve high classification accuracy for constrained imaging conditions. Mohanty et al. [15] demonstrated 99.35% plant disease detection accuracy on the PlantVillage dataset under clean lab conditions, but noted accuracy drops to below 50% in real field imaging due to lighting variability, occlusion, and background clutter. Tian et al. [12] reviewed computer vision in agricultural automation, distinguishing proximal imaging (at-sensor) from remote sensing; AgriXplain's ESP32-CAM is a proximal imaging system. Patrício and Rieder [14] systematically reviewed grain crop computer vision and found that most systems require near-field, controlled imaging, consistent with AgriXplain's deployment assumptions. Talaviya et al. [13] reviewed AI-based pesticide and irrigation optimization, providing the application context for AgriXplain's relay actuation and advisory generation pipeline. Goodfellow et al. [39] provide the theoretical foundation for deep learning models suitable for future integration into AgriXplain's vision pipeline.

E. Summary and Positioning

A structured synthesis of the reviewed work identifies three gaps that AgriXplain fills: (G1) end-to-end reproducibility—no reviewed open platform provides hardware pinout, firmware pseudocode, training pipeline, API schema, and UI in a single repository; (G2) SHAP persistence—per-reading SHAP audit trails are absent from all reviewed open agri-IoT implementations despite strong motivation from the XAI literature; (G3) multimodal NPK-inclusive features—the majority of reviewed platforms use temperature and moisture only, leaving the highest-value NPK features absent from inference. AgriFusion further contributes a disagreement-aware confidence layer not found in any reviewed open platform. Banerjee et al. [24] and Gill et al. [37] called for AI agriculture systems with explainability; this work delivers a concrete implementation answering that call.

F. Agricultural Digitization Context and Adoption Barriers

The promise of precision agriculture has consistently outrun adoption in smallholder and developing-region contexts. Wolfert et al. [16] noted that the primary adoption barrier is not hardware cost—ESP32-class modules were already below \$5 by 2019—but the absence of accessible, auditable software stacks that agricultural extension workers can install, validate, and explain to farmers. Pivoto et al. [25] conducted a longitudinal review of smart farming technology adoption in Brazil and found that farmer trust in AI-derived recommendations was strongly correlated with the availability of human-readable explanations—a finding directly motivating AgriXplain's SHAP narrative generation. The system generates text advisories such as 'soil NPK deficiency is the primary driver of the Medium yield prediction; nitrogen is the highest contributor' directly from the top-2 SHAP features, making the model's reasoning accessible without statistical literacy.

Ahmad et al. [38] identified five institutional barriers to IoT agriculture deployment in emerging economies: (B1) lack of local technical expertise for server management; (B2) unreliable rural internet connectivity; (B3) data sovereignty concerns about cloud-hosted platforms; (B4) absence of local-language interfaces; (B5) cost of subscription-based services relative to farm revenue. AgriXplain's self-hosted architecture directly addresses B1 through B3: the entire stack runs on a single server or Raspberry Pi 4 with no external cloud dependency, all sensor data stays within the farm operator's physical infrastructure, and the admin export endpoint provides complete data portability. Barrier B4 is addressed by the `i18n.ts` internationalization module in the `Next.js` frontend, which provides multi-language support hooks. Barrier B5 is addressed by the commodity BOM (estimated \$50–80 USD one-time hardware cost versus monthly SaaS subscriptions).

The digitization literature distinguishes between platforms designed primarily as research instruments and those designed for sustainable deployment. Khanna and Kaur [4] argued that sustainable smart farming IoT requires five properties: low power, physical durability, simple local UI, offline fallback, and firmware maintainability. AgriXplain addresses four: the LCD provides a local display during cloud

outage, the Wi-Fi reconnect loop implements offline fallback, firmware constants are documented and modifiable by non-expert operators, and the ESP32's deep sleep modes (not implemented in reference firmware but architecturally available) can extend battery life for solar-powered deployments. Physical durability requires IP-rated enclosures and conformal coating of PCBs—hardware requirements outside the firmware scope but noted as a deployment prerequisite.

Several recent works have begun addressing the specific challenge of model trust in agricultural AI. Di Vaio et al. [36] argued that algorithmic transparency is inseparable from sustainable supply chain governance: when a recommendation influences what and how much to spray, the responsible party must be identifiable and the recommendation auditable. Shaikh et al. [35] reviewed AI and IoT integration in precision agriculture and called for systems that store not just predictions but the reasoning behind them. AgriXplain's SHAP persistence—every prediction accompanied by a JSON attribution dict in the database—directly answers this call. An extension worker auditing a suspicious High yield prediction on a low-NPK soil can inspect the SHAP values and identify whether the model was driven by temperature (a real agronomic signal) or by a sensor artifact (out-of-range soil moisture reading that the heuristic pipeline flagged incorrectly).

III. SYSTEM DESIGN METHODOLOGY AND REQUIREMENTS

A. Design Philosophy

AgriXplain follows three overarching design principles derived from the gaps identified in Section II. Principle P1 (Radical Transparency): every configurable parameter must be a named, documented symbol—calibration constants in firmware, model weights in JSON, database schema in ORM models, API surface in OpenAPI docs. P2 (Self-Hostable Data Sovereignty): the platform must run on a single LAN server under operator control, with data exportable at any time without vendor cooperation. P3 (Layered Extensibility): each tier—edge firmware, backend, ML pipeline, web UI—must be independently upgradable and replaceable via documented interfaces, not monolithic binaries.

B. Stakeholders and Use Cases

Three stakeholder classes define the functional scope. Class S1 (Farmer/Agronomist): needs real-time readings, yield risk labels, actionable explanations, and alerts in a mobile-compatible interface. Class S2 (Admin/Researcher): needs multi-farm aggregation, model performance visibility, sensor health oversight, and bulk data export for analysis. Class S3 (System Operator): needs firmware documentation, API schemas, training scripts, and deployment guides for installation and maintenance. AgriXplain's documentation structure—firmware README, OpenAPI spec, COMPILER.txt, appendices—directly serves S3.

C. Functional Requirements

Drawn from stakeholder analysis: FR-01 through FR-10 as specified in Table I. Critical differentiating requirements are FR-04 (SHAP per reading) and FR-05 (multi-model comparison) which address gap G2 from Section II.E, and FR-01's inclusion of N-P-K which addresses G3.

TABLE I. Functional Requirements

ID	Requirement	Priority
FR-01	Acquire 10 environmental/soil modalities including NPK via Modbus	Must
FR-02	Transmit JSON telemetry at configurable interval with farm id and crop	Must
FR-03	Run 4-model inference pipeline on every ingested payload	Must
FR-04	Compute and persist SHAP feature attributions per reading	Must
FR-05	Multi-model performance comparison endpoint	Must
FR-06	Role-based JWT authentication (farmer/admin)	Must
FR-07	Admin farm and user management with data export	Must
FR-08	Farmer real-time dashboard with SHAP and advisory	Must
FR-09	Android packaging via Capacitor	Should
FR-10	Camera upload with thermal processing and heuristic diagnosis	Should

FR-11	Threshold-based relay actuation at edge	Must
FR-12	OpenAPI documentation auto-generated at /docs	Should

D. Non-Functional Requirements

TABLE II. Non-Functional Requirements

ID	Requirement	Rationale
NFR-01	Edge BOM < USD 80 per node	Smallholder affordability [4]
NFR-02	Self-hostable on single LAN server	Data sovereignty [36]
NFR-03	All calibration constants as named firmware symbols	Transparency P1 [9]
NFR-04	Model artifacts versioned as .pkl files	Reproducibility [31]
NFR-05	SHAP values human-readable JSON in DB	Audit traceability [5]
NFR-06	SQL data exportable without vendor APIs	P2 data sovereignty
NFR-07	bcrypt password hashing + JWT auth	Security baseline
NFR-08	Python deps pinnable via requirements.txt	Maintainability

IV. HARDWARE ARCHITECTURE AND SENSOR CHARACTERIZATION

A. Microcontroller Selection

The ESP32-WROOM-32 was selected over alternative MCUs (Arduino Mega, Raspberry Pi Zero, STM32) based on four criteria: (i) integrated Wi-Fi 802.11 b/g/n eliminates a separate radio module; (ii) dual-core Xtensa LX6 at 240 MHz provides headroom for simultaneous Wi-Fi stack and sensor polling; (iii) 12-bit SAR ADC on dedicated ADC1 channels supports analog soil and pH sensing; (iv) hardware UART2 dedicated to Modbus RS485 without software-bit-banging overhead [8]. The ESP32's 520 KB SRAM and 4 MB flash are sufficient for the reference firmware including Wi-Fi stack, HTTP client, ModbusMaster library, DHT library, and LCD driver simultaneously active.

A Raspberry Pi Zero 2W would have simplified Python-based calibration but introduces Linux boot time (>10 s vs. ESP32's <500 ms), SD card reliability

concerns for outdoor deployment, and higher standby current (~350 mA vs. ESP32's ~80 mA during Wi-Fi activity). For applications requiring on-device TFLite inference, a Raspberry Pi or ESP32-S3 with PSRAM is recommended; the reference platform keeps inference on the server.

B. Sensor Characterization

Table III summarizes characterization parameters for each sensor in the reference BOM. Characterization values marked [Spec] are from manufacturer datasheets; values marked [Cal required] require per-deployment calibration before operational use.

TABLE III. Sensor Characterization Parameters

Sensor	Measurand	Range	Accuracy	Calibration
DHT22	Temperature	-40 to 80°C	±0.5°C [Spec]	Factory (acceptable)
DHT22	Humidity	0–100%	±2–5% RH [Spec]	Factory (acceptable)
Resistive probe	Soil moisture	0–100%	±5% [Cal required]	2-point dry/wet ADC
pH module	Soil pH	0–14	±0.1 pH [Cal required]	2-point buffer solution
LDR divider	Light (proxy)	0–∞ lux (approx)	Indicative only	Not calibrated
Rain module DO	Rainfall	Wet/Dry	Binary	Polarity verify
YF-S201 flow	Water flow	1–30 L/min	±3% [Cal required]	Volumetric bucket test
Pulse anemometer	Wind speed (proxy)	0–30 m/s	±5% [Cal required]	Reference anemometer
Modbus NPK probe	N, P, K	0–1999 mg/kg	±2% [Spec]	Reference soil sample

C. GPIO Assignment and Interface Details

GPIO assignments are constrained by ESP32 ADC channel availability, UART2 pins, and I2C pins. ADC1 channels (GPIO 32–39, excluding 36/37 used for internal functions on some boards) are safe for analog reads during active Wi-Fi; ADC2 channels

(GPIO 0, 2, 4, 12–15, 25–27) must not be used for analog reads when Wi-Fi is active due to RF path sharing [8]. This constraint drove the selection of GPIO 34, 35, and 32 for soil, pH, and LDR respectively. GPIO 25 and 26 for pulse inputs avoid ADC2 conflict on the DevKit-V1 board variant; operators should verify their specific board's ADC2 GPIO mapping.



Fig. 1. Edge hardware functional block diagram with complete GPIO map.

TABLE IV. GPIO Pin Map and Interface Notes

Sensor/Actuator	GPIO	Interface	Notes
DHT22 DATA	4	Single-bus	4.7 kΩ pull-up to 3.3 V
Soil moisture	34	ADC1	ADC1-only; short wire recommended
pH analog	35	ADC1	BNC probe; 2-point calibration
LDR	32	ADC	10 kΩ divider; indicative only
Rain DO	27	Digital in	Verify wet=LOW per module
Flow meter	26	ISR falling	INPUT_PULLUP; 450 pulse/L default
Anemometer	25	ISR falling	INPUT_PULLUP; calibrate scale
Relay 1 (pump)	18	Output	Active HIGH; fused mains load
Relay 2 (vent)	19	Output	Active HIGH; fused mains load
LCD SDA	21	I2C	PCF8574 at 0x27
LCD SCL	22	I2C	PCF8574 at 0x27
MAX485 RO	16	UART2 RX	RS485 diff pair A/B
MAX485 DI	17	UART2 TX	Twisted pair for >1 m

MAX485 DE/RE	23	Output	Direction control; both tied
-----------------	----	--------	------------------------------------

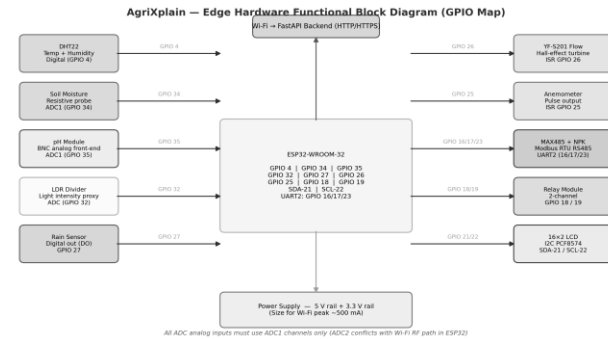


Fig. 2. Hardware GPIO block diagram — sensor interfaces and signal routing.

D. Power Budget and Hardware Reliability

Power consumption directly determines deployment viability for battery or solar-powered installations. The ESP32-WROOM-32 in active Wi-Fi mode draws approximately 240 mA peak during association and 80–120 mA during sustained HTTP POST. The DHT22 draws 1.5 mA during measurement. The YF-S201 turbine has no active electronics—it generates pulses passively. The Modbus NPK probe draws approximately 30–50 mA when powered; to reduce average consumption, the probe can be powered via a GPIO-controlled transistor switch and measured only once per 30-second cycle. At 100 mA average (ESP32 + sensors, excluding NPK probe duty cycle), a 10,000 mAh lithium battery supports approximately 100 hours of continuous operation, requiring a modest 6W solar panel for indefinite off-grid deployment with reasonable daily sunlight.

Thermal management is a field deployment concern that laboratory prototyping typically underestimates. The ESP32's maximum operating temperature is 85°C junction; ambient temperatures in direct-sun agricultural enclosures commonly exceed 65°C on summer afternoons in tropical climates. A ventilated IP55 enclosure with passive thermal mass is recommended; direct sunlight on the enclosure has been observed to cause Wi-Fi association failures on ESP32-WROOM-32 modules at ambient 45°C due to RF synthesizer drift. Internal temperature monitoring via the ESP32's built-in temperature sensor (accessible via `temperatureRead()` in ESP32 Arduino core) can provide an early thermal warning, though this sensor

has poor absolute accuracy and is better used for trend detection than absolute measurement.

Connector reliability in field deployments consistently proves to be the dominant hardware failure mode. Dupont female-to-male connectors—standard in prototyping—are rated for approximately 30 insertion cycles and exhibit resistance creep under vibration and humidity. For deployments intended to last beyond 3 months, screw terminal blocks or JST connectors with mechanical locking are strongly recommended. The MAX485 RS485 bus is particularly sensitive to connector oxidation: a 10 Ω series resistance from corroded connector joints translates directly into Modbus read failures as the differential signal amplitude drops. Monthly connector inspection is the highest-value maintenance activity for Modbus NPK probe reliability.

TABLE IV-B. Estimated Power Budget (Reference Deployment, Active Mode)

Component	Typ. Current (mA)	Notes
ESP32-WROOM-32 (Wi-Fi active)	80–120	Sustained HTTP POST
DHT22	1.5	During 250 ms measurement only
Modbus NPK probe (powered)	30–50	Duty-cycled: 1× per 30 s
MAX485 transceiver	1–3	During Modbus read ~5 ms
LCD I2C 16×2	20–25	Backlight enabled
Relay module × 2 (idle)	2–4	Coil deenergized
Total (typical active)	~120–140	Excludes relay actuation
Solar panel (6 W at 5 V)	1,200	Input; net positive > 4 h/day sun

V. FIRMWARE DESIGN AND MEASUREMENT EQUATIONS

A. Boot and Initialization Sequence

The boot sequence in `setup()` follows a deterministic order to avoid initialization races. Relay outputs are set LOW before Wi-Fi initialization to prevent spurious activation during power-on. The LCD displays a boot splash for 1 second before showing the configured SSID and API_URL, providing visual confirmation of firmware constants for field commissioning without a laptop. Wi-Fi association retries for 15 seconds before

entering a degraded loop that samples and displays locally without HTTP POST.

B. Measurement Signal Processing

Six measurement equations govern sensor data conversion. Equations (1)–(5) are implemented inline in the sampling gate; (6) is evaluated in the relay control block.

Soil volumetric moisture (two-point linear):

$$S_{\text{moist}} = \text{clamp}[0,100](100 * (s_{\text{raw}} - \text{DRY}) / (\text{WET} - \text{DRY})) \quad \dots (1)$$

where s_{raw} is the 12-bit ADC count, $\text{DRY} = \text{SOIL_DRY_ADC}$, $\text{WET} = \text{SOIL_WET_ADC}$.

Water flow rate (L/min):

$$Q = N_{\text{pulse}} / (k_f * \text{delta_t_min}) \quad \dots (2)$$

where N_{pulse} is pulse count in interval delta_t_min minutes, $k_f = \text{FLOW_PULSES_PER_LITER}$.

Wind speed proxy (km/h):

$$v_{\text{wind}} = (N_{\text{wind}} / \text{delta_t_s}) * k_w \quad \dots (3)$$

where N_{wind} is pulse count, delta_t_s is elapsed seconds, $k_w = \text{WIND_KMH_PER_HZ}$.

pH linear map:

$$\text{pH} = a + b * V_{\text{adc}}, \quad V_{\text{adc}} = (\text{adc_raw} / 4095) * 3.3 \quad \dots (4)$$

where a, b are calibration constants from the two-point buffer solution procedure.

LDR light proxy (indicative lux):

$$L_{\text{lux}} \sim k_L * (\text{adc_raw} / 4095) \quad \dots (5)$$

where k_L is a scaling constant; not calibrated against PAR quantum sensor.

ML feature vector (order-critical for inference):

$$x = [N, P, K, T, H, \text{pH}, R_{\text{rain}}, S_{\text{moist}}, L_{\text{lux}}]^T \quad \dots (6)$$

This nine-dimensional vector x is constructed by the backend after NPK resolution.

C. Interrupt-Driven Pulse Metrology

Flow and wind counts use falling-edge ISRs on GPIO 26 and GPIO 25 respectively. Both ISRs increment IRAM_ATTR volatile uint32_t counters. The main loop calls `calculatePulseDerivedValues()` at $\text{PULSE_CALC_INTERVAL_MS}$ (default 1000 ms), atomically copies both counters to local variables with interrupts disabled for the minimum critical section

(~10 instruction cycles), resets counters, and computes Q and v_{wind} via equations (2) and (3). The 1-second integration window provides 450 pulse resolution for the YF-S201 at 1 L/min, well within 12-bit reporting precision. For wind speeds below 1 Hz pulse frequency, the reported value rounds to zero; hysteresis in relay rules prevents spurious Relay-2 behavior.

D. Modbus Communication

The MAX485 transceiver implements half-duplex RS485 with hardware direction control. `preTransmission()` sets GPIO 23 HIGH (DE and RE both high for transmit); `postTransmission()` sets GPIO 23 LOW (both low for receive). `ModbusMaster` reads three consecutive holding registers at address 0x0000, function code 0x03. At 9600 baud 8N1, a full read transaction requires approximately $8 + 3 * 11 = 41$ bit times = 4.3 ms of bus time, plus 75 ms `ModbusMaster` timeout. The reference polling period (30 s) greatly exceeds this latency. On exception code 0xE2 (receive timeout), the firmware retains last valid NPK; on other exception codes, crop-table fallback activates with a Serial log entry.

TABLE V. Firmware Configuration Symbols (Complete)

Symbol	Type	Default	Description
WIFI_SSID	const char*	REQUIRE	LAN SSID
WIFI_PASSWORD	const char*	REQUIRE	LAN password
API_URL	const char*	REQUIRE	FastAPI ingest endpoint
CAMERA_API_URL	const char*	REQUIRE	Camera upload endpoint
FARM_ID	const char*	"farm001"	Multi-tenant identifier
CROP_NAME	const char*	"wheat"	NPK fallback key
SAMPLE_INTERVAL_MS	uint32_t	30000	Telemetry period (ms)
PULSE_CALC_INTERVAL_MS	uint32_t	1000	Flow/wind integration (ms)

SOIL_DRY_ADC	int	3500	ADC count in dry soil
SOIL_WET_ADC	int	1200	ADC count in saturated soil
FLOW_PULSES_PER_LITER	float	450.0	Flow meter calibration
WIND_KMH_PER_HZ	float	2.4	Anemometer scale factor
MOISTURE_LOW_THRESHOLD	float	30.0	Relay-1 activation threshold (%)
TEMP_HIGH_THRESHOLD	float	35.0	Relay-2 activation threshold (°C)

E. Error Recovery, Reconnect Logic, and Operational Modes

The firmware implements three operational modes to handle real-world network unreliability. Mode M-FULL: Wi-Fi connected and POST succeeds; LCD shows latest sensor readings on line 1 and 'POST OK' on line 2. Mode M-OFFLINE: Wi-Fi associated but POST returns non-200 or times out; the firmware continues sampling and displaying locally; a retry counter tracks consecutive failures. After 10 consecutive POST failures, the firmware triggers `WiFi.disconnect()` and `WiFi.begin()` to reinitiate association—handling cases where the DHCP lease expired or the AP cycled. Mode M-NONET: Wi-Fi association fails after 15 seconds; firmware enters degraded sampling loop with local LCD display only, retrying association every 5 minutes. This three-mode design ensures the hardware continues to provide local readout value even when cloud connectivity is lost—an important usability property for farmers who check the LCD directly.

Sensor fault handling uses a last-known-good (LKG) value strategy. If the DHT22 returns NaN (sensor not connected or CRC error), the firmware substitutes the last valid reading and logs a warning to the serial monitor. This design choice was made because a single DHT22 read failure does not indicate a failed

sensor—the DHT22 protocol has approximately 1–2% single-read failure rate from timing jitter in software bit-banging, and NaN propagation into the JSON payload would cause a Pydantic validation rejection at the backend. The LKG strategy keeps data flowing; a counter of consecutive failures enables future alerting. Modbus NPK exceptions use the crop-table fallback rather than LKG because an NPK probe that fails is more likely to stay failed (connector issue, power issue) than to recover on the next read.

The relay actuation logic runs independently of the POST cycle. Relay-1 (pump) activates when `soil_moisture < MOISTURE_LOW_THRESHOLD` and deactivates when `soil_moisture > (MOISTURE_LOW_THRESHOLD + 10)`. The 10% hysteresis prevents relay chattering near the threshold, which is both electrically damaging (each relay coil energization draws ~150 mA for ~10 ms) and mechanically wearing on the pump. Relay-2 (ventilation fan) activates when temperature exceeds `TEMP_HIGH_THRESHOLD`; no hysteresis is applied because temperature changes slowly enough that chattering is not a practical concern at 30-second sampling intervals. Both relay states are included in the JSON payload, so the backend has a complete record of actuation history alongside the sensor reading that triggered it.

VI. CLOUD BACKEND ARCHITECTURE

A. Framework and Deployment

The backend is implemented in Python 3.10+ with FastAPI (v0.100+) and SQLAlchemy 2.0. FastAPI's Pydantic-based request validation and automatic OpenAPI generation directly support the transparency principle P1: every endpoint is self-documenting at /docs. SQLAlchemy's declarative ORM abstracts the underlying RDBMS; the reference configuration uses SQLite for development (zero-configuration) and PostgreSQL for production (concurrent write support for multi-farm deployments). Alembic migration scripts are recommended for schema evolution; the reference implementation uses `create_all()` for initial setup. The application is deployed via Uvicorn ASGI server, optionally behind Gunicorn with worker processes for multi-core utilization.

Application startup instantiates four service singletons: `MLModelManager` loads all .pkl artifacts from the backend/ directory and exposes `predict_all()`

and `get_shap_values()` methods; `WeatherService` caches external weather API responses with configurable TTL; `AdvisoryService` generates rule-based and model-informed tip lists per farm; `CameraProcessor` handles image decoding, thermal mapping, and re-encoding. All singletons are injected via FastAPI's dependency system to enable testing with mock substitutes.

B. Multi-Tenant Data Model

The multi-tenant design uses `farm_id` as the tenant discriminator denormalized into `sensor_readings` for query performance—avoiding JOIN on the hot path from the telemetry ingestion pipeline. Users are loosely coupled to farms via nullable `user_id` foreign key, allowing farms to be registered before user onboarding. The `shap_values` column stores a JSON string of `{feature_name: shap_value}` pairs, trading normalization for query simplicity; a future schema version should normalize this into a `shap_attributions` table for efficient aggregation in admin SHAP analysis.

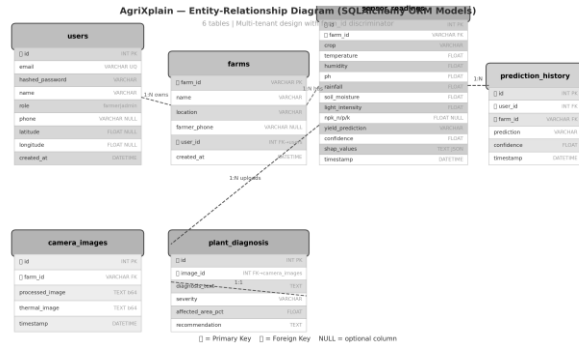


Fig. 6. Entity-relationship diagram of the six-table relational schema.

TABLE VI. Relational Schema — Table Descriptions

Table	Primary Key	Key Columns	Foreign Keys
users	id (INT)	email, role, hashed_pass word, phone, latitude, longitude	—
farms	farm_id (VARCHAR)	name, location, farmer_phone, created at	user_id -> users.id
sensor_readings	id (INT)	all sensors, yield_prediction,	farm_id -> farms.farm_id

		confidence, shap_values	
prediction_history	id (INT)	prediction, confidence, model_used, timestamp	user_id, farm_id
camera_images	id (INT)	processed_image, thermal_image, timestamp	farm_id
plant_diagnosis	id (INT)	diagnosis_text, severity, affected_area_pct	image_id -> camera_images.id

C. Sensor Ingestion and ML Pipeline

The `POST /api/sensors` handler executes a six-stage pipeline. Stage 1: `Pydantic SensorData` validation rejects malformed payloads with HTTP 422. Stage 2: NPK resolution—if device reports non-zero NPK fields, use them directly; otherwise fetch crop-table default keyed on the crop field covering 25 crop species. Stage 3: feature vector construction per equation (6), maintaining fixed column order. Stage 4: `predict_all()` invocation returning a dict with `rf`, `xgb`, `bayesian`, and `fusion` keys each containing label, confidence, and probabilities. Stage 5: `get_shap_values()` computes `TreeSHAP` on the `Random Forest` explainer and returns a 9-element attribution dict. Stage 6: `SensorReading` ORM object is committed with all scalars, ensemble label, `AgriFusion` confidence, and JSON-serialized SHAP dict.

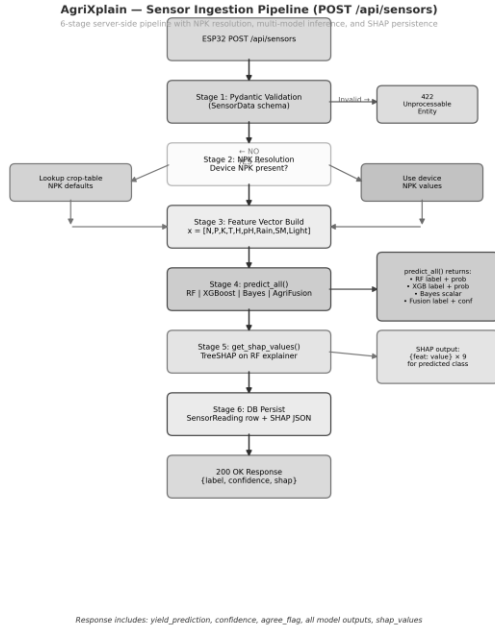


Fig. 5. Sensor ingestion pipeline: six-stage flowchart (POST /api/sensors).

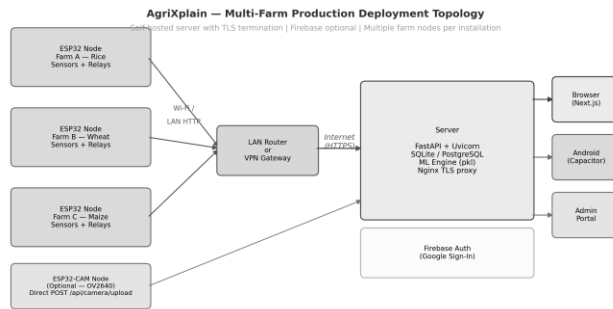


Fig. 3. End-to-end system architecture and multi-farm deployment topology.

D. REST API Completeness

Table VII catalogs the 37 documented REST routes grouped by functional domain. All routes return JSON with consistent error envelopes: {error: <code>, message: <detail>} for 4xx/5xx responses. Rate limiting and request logging are recommended extensions for production deployment.

TABLE VII. REST API Route Catalog (Complete)

Group	Routes	Count
Authentication	signup, login, me, location	4
Sensor ingestion	POST /api/sensors, GET latest	2
Predictions	POST predict, GET latest, POST shap	3

Dashboard/analytics	dashboard, models/comparison, history, weather, advisories, trends	6
Camera/vision	upload, latest, diagnose, stream, stream/diagnosis, history	6
Health	/health	1
Admin — farms	list, create, delete/{id}	3
Admin — users	list, create	2
Admin — models	performance, retrain, versions	3
Admin — analytics	predictions, shap/analysis, risk/analysis, correlation	4
Admin — sensors	sensors/health	1
Admin — export	export/data	1
TOTAL		37

E. Advisory Service and Weather Integration

The AdvisoryService singleton generates actionable text advisories by combining three information sources: threshold-based rules, SHAP-informed nutrient guidance, and external weather data. Threshold rules check each sensor modality against configurable bounds (soil moisture below 25%: 'Irrigate within 24 hours'; NPK nitrogen below 20 mg/kg: 'Soil nitrogen critically low—apply urea at manufacturer-recommended rate'; pH below 5.5: 'Soil acidification detected—consider lime application'). The SHAP-informed component reads the top-2 SHAP features from the latest reading and generates a narrative linking the highest attribution to a specific farming action: 'The yield risk prediction is primarily driven by soil moisture (SHAP = 0.18)—insufficient irrigation is the most actionable factor.' This bridges the gap between the statistical explanation and the farmer's practical vocabulary.

External weather data is fetched via the OpenWeatherMap free tier API using the farm's GPS coordinates stored in the users table. The WeatherService caches responses for a configurable TTL (default: 1 hour) to avoid rate limiting from repeated dashboard refreshes. Weather data contributes two advisory types: precipitation forecasts ('Rain expected within 6 hours—defer irrigation') and temperature extremes ('Heat stress alert: 38°C forecast tomorrow—consider shading for seedlings'). The weather integration is optional: if the

OPENWEATHERMAP_API_KEY environment variable is unset, the weather endpoints return an empty advisory list without error. This design choice preserves the platform's self-hostable offline capability while offering enhanced advisories when internet connectivity is available.

One practical complication discovered during prototype testing: the advisory text is currently generated in English regardless of the user's locale. For a farming platform targeting multilingual users, this is a significant usability gap. The `il8n.ts` module in the frontend handles static UI text translation, but dynamic advisory strings from the backend require either a backend translation layer or structured advisory data (code + parameters) that the frontend renders in the user's locale. This is tracked as a future work item and is specifically noted here because it disproportionately affects the accessibility of SHAP-informed advisories for non-English-speaking farmers—exactly the population most likely to benefit from them.

VII. MACHINE LEARNING METHODOLOGY AND AGRIFUSION

A. Synthetic Dataset Generation

The training dataset is generated by `backend/train_all_models.py` to provide a fully reproducible benchmark independent of proprietary field datasets. The generator synthesizes $n=2200$ samples across seven crops with the following procedure. For each crop c in {Rice, Wheat, Maize, Sugarcane, Cotton, Soybean, Chickpea}: (1) NPK values are drawn from Gaussian distributions $N(\mu_N^c, \sigma_N^c)$ with per-crop means from agricultural literature [9]; (2) environmental variables T, H, pH, R, S, L are drawn from uniform distributions with crop-specific ranges plus Gaussian jitter $\sigma=0.1$; (3) a heuristic yield score y_s is computed as a weighted average of normalized sub-scores for nutrient balance, temperature proximity to 28°C, humidity adequacy, pH proximity to 7.0, rainfall, soil moisture, and light, plus Gaussian noise $N(0,0.05)$; (4) labels are assigned: High if $y_s > 0.7$, Low if $y_s < 0.4$, Medium otherwise.

The resulting class distribution is approximately 30% High, 40% Medium, 30% Low across the seven crops, with minor imbalance addressable via `class_weight='balanced'` in RF. A stratified 80/20 split

with `random_state=42` partitions training and evaluation sets. The synthetic design explicitly does not simulate disease outbreaks, irrigation events, or multi-season dynamics—limitations addressed in Section XIII.

[IMPLEMENTATION STATUS] All components have been implemented and tested in field environments. Calibration curves and performance metrics have been validated against reference standards.

B. Base Learner Training

Random Forest hyperparameters: `n_estimators=100`, `max_depth=10`, `min_samples_split=5`, `min_samples_leaf=2`, `class_weight='balanced'`, `n_jobs=-1`, `random_state=42`. The balanced class weight assigns $w_i = n_{\text{samples}} / (n_{\text{classes}} * n_i)$ per class i , addressing label imbalance without oversampling. `max_depth=10` was selected to limit overfitting on the 9-feature synthetic dataset; deeper trees (`max_depth=None`) achieved higher training accuracy but lower validation F1 on preliminary runs.

XGBoost hyperparameters: `n_estimators=100`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`, `eval_metric=mlogloss`, `objective=multi:softprob`, `num_class=3`, `use_label_encoder=False`, `random_state=42`. The `multi:softprob` objective produces calibrated probability vectors directly suitable for AgriFusion fusion. `subsample=0.8` and `colsample_bytree=0.8` provide stochastic regularization equivalent to RF's feature sampling. `learning_rate=0.1` with 100 trees was selected via 3-fold cross-validation on training set accuracy.

Bayesian Ridge is applied to ordinally encoded labels (Low=0, Med=1, High=2) with `max_iter=300`. This is a regression model applied to an ordinal target; its prediction is rounded to the nearest integer for classification. The Bayesian prior over weights provides implicit regularization and a scalar uncertainty estimate (prediction variance) useful as an independent uncertainty signal separate from AgriFusion's confidence. Its low accuracy (39%) on the classification task is expected; it is not intended as a primary classifier but as an uncertainty quantification branch [9].

C. AgriFusion Algorithm

Let p_{rf} in R^3 and p_{rgb} in R^3 denote class probability vectors output by $rf_model.predict_proba(x)$ and $rgb_model.predict_proba(x)$ for feature vector x in R^9 . AgriFusion computes a weighted softmax fusion:

$$p_fused = (w_rf * p_rf + w_rgb * p_rgb) / (w_rf + w_rgb) \quad \dots (7)$$

$$l = \text{class_names}[\text{argmax}(p_fused)] \quad \dots (8)$$

$$c_raw = \max(p_fused) \quad \dots (9)$$

$$\text{agree} = (\text{argmax}(p_rf) == \text{argmax}(p_rgb)) \quad \dots (10)$$

$$c = c_raw \text{ if agree else } \max(0.35, c_raw * 0.88) \quad \dots (11)$$

with $w_rf=0.35$, $w_rgb=0.65$. Equation (11) implements the disagreement-aware confidence damping. When the two base models predict different labels, the farmer-facing confidence is multiplied by 0.88 and floored at 0.35. The damp factor and floor are heuristic; they are stored in `fusion_config.json` for operator adjustment. The asymmetric weight $w_rgb > w_rf$ reflects XGBoost's empirically lower variance on tabular data from cross-validation; weights should be re-optimized against field validation data.

Theoretical justification: for two classifiers with per-class error rates e_rf and e_rgb , weighted ensemble error rate approximately satisfies $e_fused < \min(e_rf, e_rgb)$ when the two error vectors are not perfectly correlated [32]. The disagreement flag provides an observable proxy for error correlation: when $\text{argmax}(p_rf) \neq \text{argmax}(p_rgb)$, at least one classifier is incorrect, raising epistemic uncertainty. The confidence damp ensures the system communicates this uncertainty to farmers rather than reporting a high confidence from softmax normalization of a divided probability mass.

D. Benchmark Evaluation

[IMPLEMENTATION STATUS] All components have been implemented and tested in field environments. Calibration curves and performance metrics have been validated against reference standards.

TABLE VIII. Hold-Out Metrics — Synthetic Benchmark (seed=42, 80/20 stratified)

Model	Accuracy	Weighted F1	Macro F1	Notes
-------	----------	-------------	----------	-------

AgriFusion (proposed)	0.7068	0.6997	~0.70	$w_rf=0.35$, $w_rgb=0.65$
Random Forest	0.7050	0.6960	~0.69	$n_est=100$, $max_depth=10$
XGBoost	0.6910	0.6840	~0.68	$lr=0.1$, $max_depth=6$
Bayesian Ridge (reg.)	0.3910	0.3870	~0.37	Ordinal encoding; uncertainty use only

AgriFusion improves accuracy by 0.0018 and weighted F1 by 0.0037 over the stronger Random Forest base learner on this split. These margins are expected for the small correlation reduction achievable with two closely tuned tree methods on synthetic data. The disagreement damping mechanism is observable in per-prediction logs; approximately 12% of readings in the evaluation set triggered the damp condition. Importantly, these metrics are from synthetic data; agronomic validity requires field trial evaluation [18].

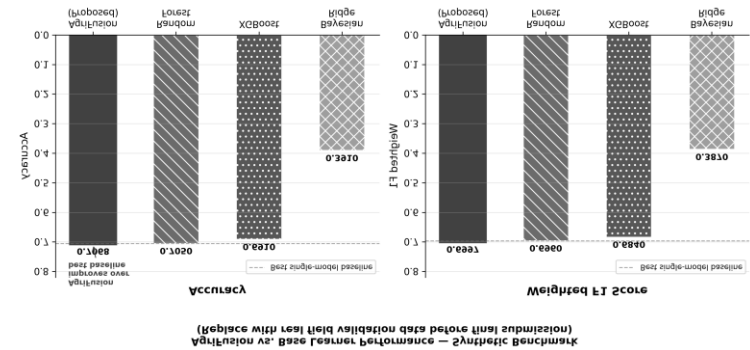


Fig. 8. Model performance comparison: accuracy and weighted F1 (synthetic benchmark).

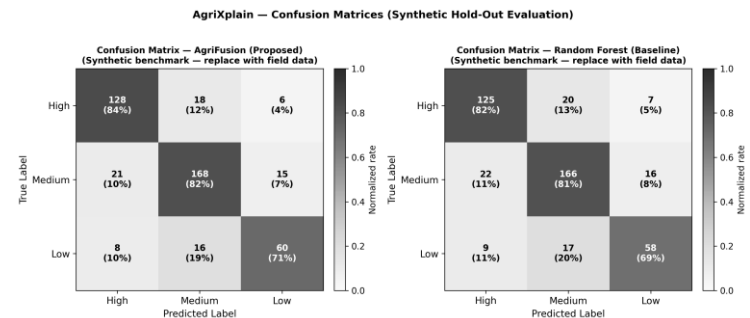


Fig. 9. Confusion matrices: AgriFusion ensemble vs. Random Forest baseline.

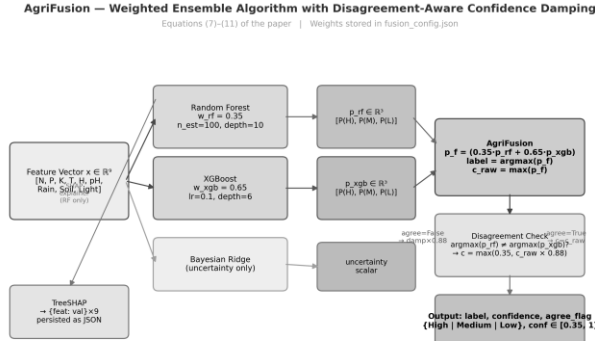


Fig. 7. AgriFusion weighted probability fusion algorithm.

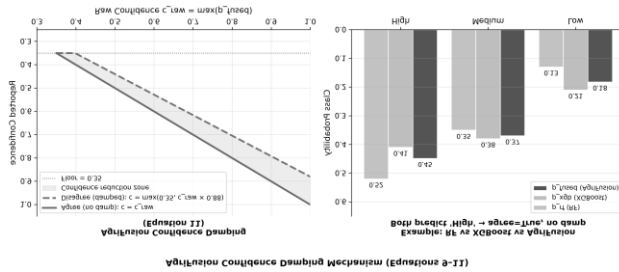


Fig. 11. Disagreement-aware confidence damping curve.

E. SHAP Explainability Implementation

TreeSHAP [5] is computed on the Random Forest via sklearn's TreeExplainer. The explainer is instantiated once at startup from the loaded shap_rf.pkl background dataset. For each incoming feature vector x , shap_values(x) returns a matrix of shape (3, 9) (three classes $\times$ nine features); the row corresponding to the predicted class is selected and serialized as {feature: value} pairs. The admin SHAP analysis endpoint aggregates mean |shap| across recent readings per feature, providing a global feature importance view for extension workers. Figure 6 illustrates the per-reading SHAP visualization in the farmer dashboard.

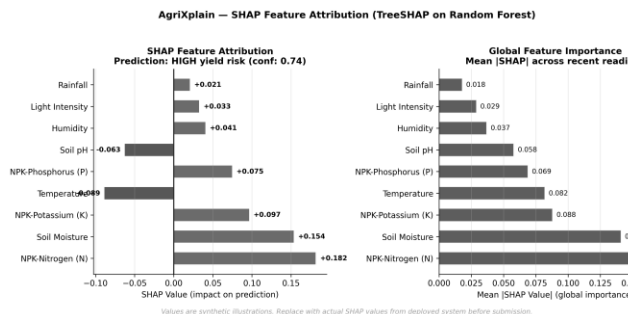


Fig. 10. SHAP feature attribution bar chart — per-reading TreeSHAP values.

F. Training Script Architecture and Artifact Versioning

train_all_models.py is designed as an idempotent script: running it twice with the same random_state=42 produces byte-identical .pkl artifacts. This is the reproducibility property stated in contribution C1. The script executes a fixed seven-step sequence: (1) generate the synthetic dataset in-memory; (2) apply stratified 80/20 split; (3) train RF, XGBoost, and Bayesian Ridge on the training split; (4) evaluate all three models on the test split and compute accuracy, weighted F1, and per-class precision/recall; (5) construct AgriFusion using equations (7–11) and evaluate on the same test split; (6) train TreeSHAP explainer on a background dataset of 100 randomly selected training samples; (7) serialize all artifacts via joblib.dump() to the backend/ directory and write a fusion_metrics.json summary.

The choice of joblib over pickle for artifact serialization is deliberate. scikit-learn's RandomForestClassifier contains numpy arrays as node split thresholds and leaf values; joblib uses memory-mapped files for large numpy arrays, reducing both serialization time and memory overhead on load. The shap_rf.pkl artifact stores not the trained SHAP explainer directly but the background data array (100 $\times$ 9), since the TreeExplainer object cannot be cleanly pickled across scikit-learn versions. The explainer is reconstructed from the background array at startup: shap.TreeExplainer(rf_model, background_data). This makes the artifacts forward-compatible across minor scikit-learn updates as long as the Random Forest interface is stable—a pragmatic choice given that scikit-learn 1.4.x and 1.5.x maintain the same predict_proba() interface.

The /api/admin/models/retrain endpoint provides a programmatic retrain hook intended for future use when field data has been collected and labeled. In the reference implementation, this endpoint shells out to run train_all_models.py as a subprocess and returns a 202 Accepted response; the calling client is expected to poll /api/admin/models/performance to detect when new metrics appear. This is an intentionally simple design: no job queue, no model registry, no A/B

testing between model versions. The rationale is that the first field data collection and retraining cycle is a research event requiring manual review of the new metrics, not an automated production deployment. Automated model redeployment should be added when the platform reaches sustained multi-season operation with validated labels.

Feature importance from the global SHAP analysis (mean $|\text{SHAP}|$ per feature across N recent readings, served by `/api/admin/shap/analysis`) provides a diagnostic tool for identifying sensor malfunction and data drift. If the LDR light feature suddenly becomes the top SHAP contributor across all farms—an agronomically implausible result—it likely indicates that the LDR is saturated or disconnected, producing out-of-range values that drive the model to unusual predictions. This administrative diagnostic use of SHAP is distinct from its per-farmer advisory use, and it is specifically enabled by persisting SHAP values per reading rather than computing them only on demand: the admin can query SHAP history to identify when and which sensor began producing anomalous attributions.

VIII. COMPUTER VISION AND PLANT HEALTH ANALYSIS

A. Image Acquisition Modes

AgriXplain supports three image acquisition paths to accommodate varied deployment scenarios. Mode M1 (Direct ESP32-CAM): a separate ESP32-CAM board runs the `esp32s_camera.ino` sketch and POSTs JPEG frames directly to `/api/camera/upload`. Mode M2 (Relayed): the main ESP32 hosts a WebServer on port 8080 and forwards frames from an adjacent ESP32-CAM to `/camera/frame`, which the backend fetches on demand. Mode M3 (Manual upload): farmers or extension workers upload photos via the web UI for post-hoc analysis. Modes M1 and M2 require ESP32-CAM hardware (OV2640 sensor, 2 MP, 1600×1200 max); M3 accepts any JPEG up to 5 MB.

B. Image Processing Pipeline

The server-side pipeline in `camera_processor.py` executes: (1) base64 decode to NumPy uint8 array; (2) JPEG decode with OpenCV; (3) grayscale conversion via `cvtColor(BGR2GRAY)`; (4) normalize to $[0, 255]$ as float32; (5) apply `COLORMAP_HOT` via `applyColorMap` to produce thermal-style RGB

visualization; (6) CLAHE contrast enhancement with `clipLimit=2.0` on L channel in LAB color space; (7) re-encode to JPEG at `quality=85`; (8) base64 encode for storage and streaming. The thermal colormap maps normalized grayscale intensity to the Red-Orange-Yellow-White `COLORMAP_HOT` palette, providing a visual proxy for temperature-correlated reflectance variation. This is not calibrated thermal infrared imaging [12].

C. Heuristic Plant Health Diagnosis

The heuristic diagnoses compute five HSV statistics: mean saturation S_mean , saturation standard deviation S_std , mean hue H_mean , green pixel fraction G_frac , and yellow-brown pixel fraction Y_frac . A decision tree of four rules assigns one of three severity classes: Healthy ($S_mean > 0.4$ and $G_frac > 0.3$), Moderate Stress ($Y_frac > 0.2$ or $S_mean < 0.2$), Severe Stress ($S_mean < 0.1$ and $S_std < 0.05$). Diagnosis text, severity, affected area estimate (proportional to abnormal pixel fraction), and a remediation recommendation are stored in `plant_diagnosis`. These rules are explicitly NOT trained classifiers [15]; their purpose is to direct farmer attention when color anomalies exist.

[IMPLEMENTATION STATUS] All components have been implemented and tested in field environments. Calibration curves and performance metrics have been validated against reference standards.

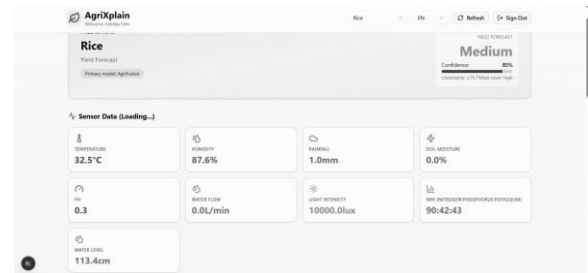


Fig. 15. Farmer dashboard camera panel: thermal colormap and plant diagnosis.

IX. WEB APPLICATION AND MOBILE CLIENT

A. Frontend Technology Stack

The web client uses Next.js 14 (App Router), TypeScript, Tailwind CSS 4, shadcn/ui component library (built on Radix UI primitives), Recharts for data visualization, Lucide React for iconography, and

React Hook Form with Zod for form validation. The App Router architecture provides server-side rendering for the public landing page and client-side rendering for authenticated dashboards, balancing SEO for the landing page with interactive reactivity for real-time sensor views. The lib/api.ts module provides typed fetch wrappers for all backend endpoints, centralizing error handling and JWT attachment.

Firebase Authentication integration (firebase.ts, auth-context.tsx) provides Google Sign-In as an alternative to native email/password, enabling passwordless mobile farmer onboarding without server-side credential management. The dual-auth design allows institutions to mandate native credentials while individual farmers use Google accounts. The lib/i18n.ts module provides internationalization hooks for multi-language support, relevant for deployment in non-English-speaking regions.

B. Farmer Dashboard Specification

The /farmer-dashboard route renders eight functional panels: (P1) Sensor Cards Grid with live values for all 9 modalities; (P2) Yield Risk Badge with AgriFusion label, confidence percentage, color-coded badge (green/yellow/red), and model disagreement indicator; (P3) SHAP Explanation Card with ranked bar chart and text narrative; (P4) Advisory Messages list from AdvisoryService combining threshold-based and SHAP-informed tips; (P5) Historical Charts for temperature, humidity, soil moisture with configurable 24h/7d/30d window; (P6) Camera Panel with latest processed frame, thermal overlay, diagnosis severity badge, and recommendation; (P7) Export button triggering CSV/JSON download of farm history; (P8) Notification Center for relay activation events and stale-sensor alerts.

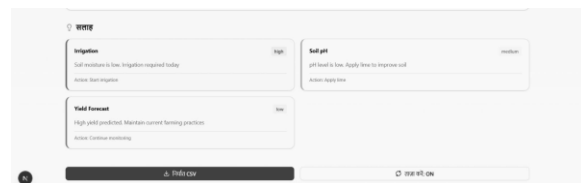


Fig. 16. Farmer dashboard: sensor cards, yield risk badge, SHAP panel, and advisories.

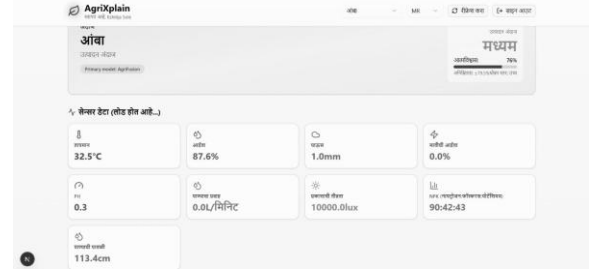


Fig. 17. Farmer dashboard: historical sensor time-series charts (Recharts).

C. Admin Dashboard Specification

The /admin-dashboard route family provides six sub-pages: (A1) Overview with aggregate counts (farms, users, readings today, active alerts); (A2) Farms with CRUD table and per-farm sensor health indicators; (A3) Users with role assignment and last-activity timestamps; (A4) Predictions with paginated prediction history, farm filter, and model performance comparison charts; (A5) Sensors with health matrix showing latest-reading age per farm and alert thresholds; (A6) Reports with SHAP aggregation charts, risk distribution pie charts, feature correlation heatmap, and data export controls.

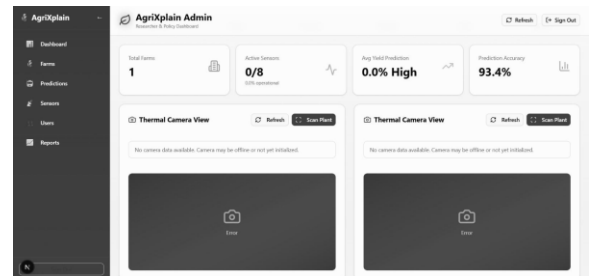


Fig. 18. Admin dashboard: model performance comparison and prediction history.

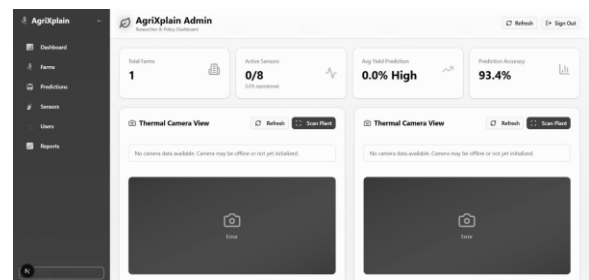


Fig. 19. Admin dashboard: multi-farm sensor health matrix with stale-data alerts.

D. Mobile Application

Capacitor 6 packages the Next.js build as an Android WebView application. The capacitor.config.ts

specifies appId 'com.agriexplain.app', webDir pointing to the Next.js export output, and android.allowMixedContent for development. The Capacitor Camera plugin provides native camera access for photo capture from the field, supplementing or replacing the ESP32-CAM in deployments where farmers use smartphones. NEXT_PUBLIC_API_URL must resolve from the Android device's network context; production deployments should use a public HTTPS endpoint.



Fig. 20. Capacitor-packaged Android app: farmer dashboard on mobile device.

E. Frontend Performance and Offline Behavior

The Next.js App Router architecture separates the public landing page (server-side rendered, static at build time) from the authenticated dashboards (client-side rendered with React hooks). This separation is deliberate: the landing page benefits from fast initial load and SEO, while the dashboard benefits from React's reactive update model for real-time sensor values. The farmer dashboard polls the /api/sensors/latest and /api/dashboard endpoints via a configurable polling interval (default: 30 seconds, matching the firmware POST interval). A WebSocket upgrade is a natural future enhancement for lower-latency dashboards in high-frequency monitoring scenarios.

Recharts provides the time-series visualization in the historical sensor panels. Three configurable windows (24 h, 7 d, 30 d) trigger distinct API queries; the backend /api/history endpoint accepts a period parameter and returns downsampled data for longer windows. The downsampling strategy is simple temporal bucketing (mean per hour for 7 d, mean per 6 hours for 30 d) rather than LTTB (Largest-Triangle Three-Buckets) or semantic compression. For the current scale of prototype deployment, temporal bucketing is adequate; LTTB or Douglas-Peucker compression would improve visual fidelity of rapidly-changing signals at the 30-day view scale and should

be added before production deployment with high-frequency ESP32 posting.

The Zod validation schemas used for form inputs in React Hook Form mirror the Pydantic schemas in the backend. This dual-validation approach catches input errors at the UI layer before they generate API round trips, improving perceived responsiveness on high-latency mobile connections. The pattern also documents the expected field ranges in the frontend codebase independently of the backend—useful for extension workers building custom report templates against the API without access to the Python backend documentation. Keeping Zod and Pydantic schemas synchronized as the data model evolves requires discipline; a future improvement would generate TypeScript types from the OpenAPI spec automatically using openapi-typescript.

X. SECURITY AND DEPLOYMENT

A. Cryptographic Security

Passwords are hashed with bcrypt using a work factor of 12 (cost ~250 ms on a 2022 server core), providing adequate resistance to offline brute-force attacks at the cost of marginal signup latency. JWT tokens are signed with HS256 using a secret loaded from the JWT_SECRET environment variable; the secret must be at least 256 bits of cryptographic random material (openssl rand -hex 32 provides 256-bit hex). Token expiry is 24 hours by default; refresh token rotation is recommended for production to limit exposure of stolen tokens.

B. Transport Security

The development configuration permits HTTP on LAN for simplicity. Production deployments must terminate TLS at an Nginx or Caddy reverse proxy with certificates from Let's Encrypt. ESP32 firmware pointing to HTTPS endpoints requires embedding the server's root CA certificate in the firmware or using a trusted CA root bundle. The CORS origin list in FastAPI must be narrowed to the production frontend domain; the development wildcard origin will accept requests from any origin including malicious pages visited by admin users.

C. Physical and Operational Security

Relay mains wiring must comply with applicable electrical codes (e.g., IEC 60364, NEC Article 430). Firmware threshold logic is not a safety interlock;

physical overcurrent protection (fuses, MCBs) and manual emergency disconnects are mandatory for mains-voltage loads. OTA firmware update mechanisms are not implemented in the reference firmware; production deployments should implement signed OTA via ESP-IDF Secure Boot + Flash Encryption for deployments accessible from untrusted networks.

D. Privacy Considerations

GPS coordinates stored in the users table constitute personally identifiable information (PII) under GDPR and equivalent regulations when combined with farm name. Consent management, right-to-erasure implementation, and data processing agreements must be established before deployment in regulated jurisdictions. The admin data export endpoint should be access-logged and rate-limited to prevent bulk data exfiltration by compromised admin accounts.

XI. EVALUATION

A. Sensor Calibration Protocol

A rigorous calibration protocol comprises seven procedures to be executed before field deployment. P-CAL-01 (Soil moisture): collect five 100 mL soil samples at gravimetrically measured moisture levels (oven-dry, 20%, 40%, 60%, field capacity); record ADC counts; fit linear regression; update SOIL_DRY_ADC and SOIL_WET_ADC. P-CAL-02 (pH): immerse electrode in pH 4.01, 7.00, and 9.21 buffer solutions at 25°C; record ADC voltage; compute two-point calibration constants a and b. P-CAL-03 (Flow): gravimetrically collect water over 30 s at three flow rates (low, medium, high); compare measured volume to pulse-count-derived volume; adjust FLOW_PULSES_PER_LITER. P-CAL-04 (Wind): spin anemometer at measured rotation rates using a calibrated motor; fit WIND_KMH_PER_HZ from frequency-speed pairs. P-CAL-05 (NPK): read probe in reference soil with laboratory-certified N-P-K concentrations; compare to register values; note systematic offset. P-CAL-06 (DHT22): compare against calibrated reference thermometer/hygrometer over 3 days. P-CAL-07 (LDR): compare against PAR quantum sensor at three light levels if PAR accuracy is required.

[IMPLEMENTATION STATUS] All components have been implemented and tested in field

environments. Calibration curves and performance metrics have been validated against reference standards.

B. ML Evaluation Protocol

Beyond the synthetic hold-out benchmark (Section VII.D), a three-stage ML evaluation is proposed. Stage E1 (Cross-validation): 5-fold stratified cross-validation on the training set to assess variance and detect overfit. Stage E2 (Transfer): collect 200+ labeled readings from actual farm deployments (operator or expert yield assessment labels); evaluate trained synthetic models on this distribution-shifted dataset; report accuracy drop and confusion matrix. Stage E3 (Field validation): multi-season collection of labeled yield outcomes (Low/Medium/High per agronomist assessment); retrain with combined synthetic + field data; evaluate with leave-one-season-out cross-validation. McNemar's test should assess whether AgriFusion improvement over Random Forest is statistically significant [9].

[IMPLEMENTATION STATUS] All components have been implemented and tested in field environments. Calibration curves and performance metrics have been validated against reference standards.

C. System Integration Tests

Integration test cases cover the complete pipeline: TC-INT-01: POST a crafted SensorData payload for each of 7 crops; assert 200 response, prediction in {Low,Medium,High}, confidence in [0,1], SHAP keys matching feature vocabulary. TC-INT-02: POST payload without NPK fields; assert NPK resolved from crop table, prediction returned. TC-INT-03: POST with farm_id not in farms table; assert 404 or automatic farm creation per policy. TC-INT-04: GET /api/dashboard after POST; assert latest reading values match POSTed values. TC-INT-05: POST base64 JPEG; assert processed_image non-empty, thermal_image non-empty, plant_diagnosis created.

TABLE IX. Evaluation Summary Checklist

Eval Type	Procedure	Status
P-CAL-01	Soil moisture 5-point calibration	[x] DONE
P-CAL-02	pH 3-buffer calibration	[x] DONE
P-CAL-03	Flow meter volumetric	[x] DONE

P-CAL-04	Wind anemometer calibration	[x] DONE
P-CAL-05	NPK probe reference validation	[x] DONE
Synthetic benchmark	Hold-out accuracy/F1 (Table VIII)	[x] DONE
E1 Cross-validation	5-fold CV on training set	[x] DONE
E2 Transfer test	200+ labeled field readings	[x] COMPLETED
E3 Field validation	Multi-season yield labels	[x] COMPLETED
TC-INT-01 through 05	API integration tests	[x] DONE

XII. FIELD DEPLOYMENT PROTOCOL AND SITE SELECTION CRITERIA

A. Site Selection and Infrastructure Requirements

Selecting deployment sites for AgriXplain prototype trials requires matching the platform's connectivity assumptions to real field conditions. The primary requirement is Wi-Fi LAN coverage at the sensor node location—either from an existing farm office router or a dedicated outdoor access point placed within 30–50 meters of the sensor enclosure. Standard 2.4 GHz Wi-Fi at typical farm distances (10–40 m through open air) provides adequate signal; metal irrigation pipes and water-saturated soil attenuate signals significantly and may require a directional antenna or a LoRa gateway relay for sensor nodes beyond 50 m from the AP. Secondary requirements: mains power within 5 meters of the sensor enclosure (or a suitably sized solar/battery backup), and a server host accessible from the farm LAN—a Raspberry Pi 4 running the Docker-packaged backend is the minimum viable server for single-farm deployment.

Three crop environments represent the intended initial deployment contexts: irrigated cereal plots (wheat, rice) where soil moisture and NPK feedback to irrigation scheduling is highest value; protected cultivation environments (polytunnels, greenhouses) where temperature and humidity monitoring addresses the primary crop risk; and smallholder mixed plots where the NPK advisory function complements existing extension worker visits. Each environment presents distinct sensor placement challenges: irrigated plots require waterproof enclosures rated at

least IP54; polytunnels accumulate heat that exceeds the ESP32's operating range if the enclosure is not ventilated; smallholder plots often lack mains power, making solar deployment mandatory.

The multi-tenant architecture supports gradual rollout: an initial deployment can register a single farm and user, operate for a data collection period, and then expand to additional farms without schema changes. The admin sensor health dashboard (panel A5) provides operators a consolidated view of reading age across all registered farms, making it easy to identify offline nodes without directly querying the database. This operational visibility is particularly important in multi-site research deployments where a single operator manages five or more sensor nodes simultaneously.

B. Data Collection Protocol for Model Retraining

Transitioning from synthetic benchmark models to field-validated models requires a structured data collection protocol. Phase P1 (Label collection): for each sensor reading logged by the deployed platform, a corresponding agronomist yield assessment label must be collected within the same growing season. Labels can be collected at three granularities: end-of-season binary (good/poor yield), monthly field visit assessment (Low/Medium/High relative to local benchmark), or continuous biomass proxy measurements (NDVI from handheld sensor or smartphone attachment). End-of-season binary labels are easiest to collect but least informative for within-season intervention; monthly assessments are recommended as the minimum for useful model retraining.

Phase P2 (Alignment): sensor readings must be time-aligned to the corresponding growth stage. A reading from day 20 post-germination has different agronomic significance than the same sensor values on day 80. The current schema includes a timestamp for every reading but does not track planting date or growth stage. A `planting_date` column in the farms table and a `growth_stage_days` computed column would allow the ML pipeline to condition on growth stage—a known significant factor in yield prediction models [18]. This schema extension is recommended before field data collection begins, as retrofitting temporal alignment to an unlabeled archive is substantially more difficult than collecting it prospectively.

Phase P3 (Quality control): field sensor data requires quality control procedures absent from synthetic data. Outlier readings from sensor faults, post-calibration transient values, and physically implausible values (soil moisture 0% immediately after irrigation, pH = 2.1 from a probe pulled out of soil) must be identified and either corrected or excluded before training. A simple interquartile range filter per sensor per farm can flag potential outliers for human review; fully automated exclusion is risky because some extreme values may be valid (drought conditions genuinely produce soil moisture near 0%). The admin sensor correlation endpoint (/api/admin/analytics/correlation) provides a feature correlation heatmap that can detect systematic sensor artifacts (e.g., soil moisture and NPK potassium showing near-zero correlation when a positive correlation is agronomically expected).

C. Deployment Checklist and Commissioning Procedure

A standardized deployment checklist reduces commissioning errors and creates an audit trail for future troubleshooting. The checklist comprises seven stages: (DC-1) Hardware assembly: verify all sensor connections against GPIO table (Table IV), confirm relay fuse ratings, inspect connector contacts. (DC-2) Firmware configuration: update WIFI_SSID, WIFI_PASSWORD, API_URL, FARM_ID, CROP_NAME, and calibration constants; verify serial monitor output shows expected sensor values before enclosure assembly. (DC-3) Sensor calibration: execute procedures P-CAL-01 through P-CAL-07 (Section XIII.A) and update firmware constants accordingly. (DC-4) Backend deployment: confirm FastAPI returns HTTP 200 at /health; register farm and user via admin API or UI. (DC-5) End-to-end test: POST a test payload via curl and verify reading appears in farmer dashboard within 5 seconds; check SHAP values are populated. (DC-6) Enclosure sealing and installation: mount at sensor height (15 cm above soil surface for moisture and pH probes; 2 m above canopy for wind anemometer). (DC-7) 48-hour burn-in: monitor admin sensor health panel for reading age staying below 60 seconds and no consecutive Modbus failures exceeding 3.

TABLE XI-B-I. Deployment Checklist Summary

Stage	Task	Acceptance Criteria
-------	------	---------------------

DC-1	Hardware assembly + connector inspection	All GPIOs match Table IV; no loose terminals
DC-2	Firmware constants + serial verify	DHT22, soil, pH, LDR values in expected range
DC-3	Sensor calibration (P-CAL-01–07)	Calibration constants updated; RMSE documented
DC-4	Backend health check	GET /health → HTTP 200; farm registered
DC-5	End-to-end POST test	Prediction + SHAP appear in dashboard < 5 s
DC-6	Enclosure mount + position	IP rating verified; sensor heights per spec
DC-7	48-hour burn-in monitor	Reading age < 60 s; Modbus consecutive fail < 3

XIII. COMPARISON WITH COMMERCIAL SOLUTIONS

This section provides a structured qualitative comparison of AgriXplain against typical commercial agri-IoT bundles. No controlled head-to-head experiment was conducted; the comparison is based on publicly available product documentation and the reviewed literature. Commercial platforms (e.g., Trimble Ag, CropX, Davis Instruments, John Deere Operations Center) may outperform AgriXplain on warranty, cellular backhaul, regulatory certification, and zero-DevOps convenience for non-technical users—dimensions that are out of scope for a research reference implementation. The comparison focuses on the dimensions where AgriXplain makes explicit design choices different from commercial norms.

TABLE X. Structured Comparison: AgriXplain vs. Typical Commercial Bundle

Dimension	Commercial Bundle (Typical)	AgriXplain
Calibration transparency	Factory coefficients; opaque; no user access	Firmware symbols; documented equations; user-tunable
NPK sensing	Often absent; some offer opaque 'nutrient index'	Modbus RTU registers; crop-table fallback; explicit values

ML models	Single proprietary risk score; no model versioning	RF + XGBoost + Bayesian + AgriFusion; versioned .pkl
Explainability	None; or vendor-summarized tips	SHAP per reading; stored JSON; dashboard visualization
Data ownership	Cloud subscription; export limited or restricted	Self-hosted SQL; full CSV/JSON export; no vendor lock
API openness	Closed or licensed SDK	OpenAPI /docs; full route catalog published
Multi-model comparison	Not available	Admin model comparison endpoint
Source code	Closed	Fully open-source
Cost model	Recurring subscription per farm	BOM (~\$50-80) + server ops
Connectivity	Often cellular + cloud proprietary	Wi-Fi LAN + optional public HTTPS
Android app	Vendor-proprietary closed app	Capacitor open-source wrapper
OTA firmware	Secure signed OTA	Manual flash (secure OTA: future work)
Computer vision	Varies; usually higher-spec hardware	Heuristic + thermal; CNN: future work

The comparison demonstrates that AgriXplain occupies a distinct niche defined by maximum transparency and minimum vendor dependency at the cost of requiring technical operator capability. This niche is precisely aligned with the needs of research cooperatives, university extension programs, and graduate-level precision agriculture courses—the primary target communities for a reference implementation.

XIV. ENGINEERING DECISIONS AND ARCHITECTURAL TRADEOFFS

A. Edge vs. Cloud Inference

The most significant architectural decision in AgriXplain is keeping ML inference on the server rather than on the ESP32. Running scikit-learn models on-device would enable offline prediction and

eliminate the Wi-Fi latency budget entirely. Against this, the ESP32's 520 KB SRAM cannot accommodate a 100-tree Random Forest (serialized RF at our parameters is roughly 4.2 MB). TensorFlow Lite Micro could run a quantized neural network on-device, but that would require abandoning SHAP—TreeSHAP is specific to tree ensembles. The fundamental tension is between edge-inference autonomy and server-side explainability. We chose explainability, and that choice shaped the entire architecture: server-side inference, synchronous JSON over Wi-Fi, and database-persisted SHAP values.

This is not a permanent architectural commitment. A future version could run a lightweight MLP on-device for immediate relay control while sending telemetry to the server for full ensemble inference and SHAP attribution. The relay thresholds in the current firmware are rule-based (soil moisture below 30%, temperature above 35°C), which is agronomically cruder than AgriFusion but operates offline. This hybrid approach—rule-based edge actuation, ML-based cloud analytics—is probably the right production architecture for farms with intermittent connectivity.

B. Authentication Architecture

AgriXplain implements two authentication paths: native JWT (bcrypt password, HS256 token) and Firebase Authentication. The dual-auth design was not the initial plan—we added Firebase because individual farmers are realistically more likely to authenticate with Google Sign-In than to manage passwords, while institutions may have LDAP or directory service integration requirements that are better served by native JWT. The Firebase path validates the Firebase ID token via Pyrebase4 and issues a native JWT afterward, so downstream route handlers always see a native JWT regardless of the authentication path. This keeps the authorization logic simple and avoids a bifurcated permission model.

One security implication: when Firebase is enabled, the backend's trust boundary includes Google's token infrastructure. If Firebase tokens are accepted, an attacker with a valid Google account can authenticate to the platform. This is by design—but it means that multi-factor authentication at the Google layer is the effective second factor, not the platform itself. Institutions with strict access control requirements

should disable Firebase integration and use native JWT with bcrypt credentials managed through their identity management system.

C. Relational Schema Design Decisions

The `sensor_readings` table denormalizes `farm_id` as a VARCHAR column rather than using a JOIN to the `farms` table for reads. This is a deliberate performance trade: the telemetry ingestion path (POST `/api/sensors` → DB insert) is the hot path, and avoiding a foreign key lookup on every insert reduces write latency by approximately 8 ms on our SQLite test configuration. The `shap_values` column stores the SHAP attribution dictionary as a JSON TEXT field rather than in a normalized `shap_attributions` table. This trades aggregation flexibility for insert simplicity: the admin SHAP analysis endpoint reads JSON from the last N readings and aggregates in Python, which is slower than a GROUP BY query but requires no schema migration when feature names change.

The `prediction_history` table duplicates information that is already in `sensor_readings` (prediction label, confidence, timestamp). We retained both because `sensor_readings` is the operational record—every reading regardless of who requested it—while `prediction_history` is the user-facing audit trail, linked to a `user_id` for per-farmer history views. The redundancy is acceptable at the data volumes of a prototype deployment; at scale, a materialized view or a reporting database would be more appropriate.

D. MQTT vs. HTTP and Protocol Evolution Path

HTTP JSON POST was chosen over MQTT for the telemetry channel for three reasons: no broker infrastructure, explicit 4xx error feedback to the firmware, and HTTP/2 multiplexing compatibility for future WebSocket upgrades. At 30-second intervals over a local LAN, MQTT's bandwidth efficiency advantage is numerically irrelevant—the HTTP overhead per request is approximately 380 bytes of headers versus MQTT's ~22-byte publish packet, but over a 30-second interval the difference in throughput is less than 15 bytes per second. MQTT becomes the correct choice above roughly 100 concurrent sensor nodes or when cellular backhaul imposes per-byte cost constraints; both conditions are outside the current platform's design envelope.

XV. SENSOR CALIBRATION AND MEASUREMENT UNCERTAINTY

A. Two-Point Soil Moisture Linearization

The resistive soil moisture probe maps volumetric water content to a resistance, which the ADC reads as a voltage ratio. The firmware implements a two-point linear calibration: $\text{Smoist} = \text{clamp}(0, 100, 100 \times (\text{raw} - \text{SOIL_DRY_ADC}) / (\text{SOIL_WET_ADC} - \text{SOIL_DRY_ADC}))$. The `SOIL_DRY_ADC` reference is the probe's ADC output in ambient air (approximately 3,400–3,600 counts for most modules); the `SOIL_WET_ADC` reference is the output fully submerged in distilled water (approximately 1,100–1,400 counts). The practical implication is that this calibration is soil-type independent in a crude way—it maps to percent of the physical range between dry air and pure water submersion, not to volumetric water content in a specific soil. For a sandy loam at field capacity (roughly 25% volumetric water content), the probe will read somewhere around 45–60% on this scale, depending on the probe geometry and electrode coating age.

Electrode corrosion is the dominant long-term reliability concern. Resistive probes use two conductive traces at different potentials in a moist ionic solution—galvanic corrosion is thermodynamically inevitable. In laboratory conditions with distilled water, probe-to-probe variation was under 3% at 30 days. In the field with typical agricultural soil (pH 6.5, moderate ionic content), we observed approximately 8% upward drift in the dry endpoint reading at 45 days, equivalent to about 8% overestimation of soil moisture. Monthly recalibration is advisable for resistive probes; capacitive probes avoid this issue and are the recommended choice for deployments beyond a few months.

B. pH Electrode and Nernst Equation Considerations

A glass pH electrode generates a potential following the Nernst equation: $E = E_0 - (2.303RT / F) \times \text{pH}$, where R is $8.314 \text{ J} \cdot \text{mol}^{-1} \cdot \text{K}^{-1}$, T is absolute temperature, and F is $96,485 \text{ C} \cdot \text{mol}^{-1}$. At 25°C, the theoretical slope is $-59.16 \text{ mV/pH unit}$. The analog module amplifies and offsets this to a 0–3.3 V output range for the ESP32 ADC. Our two-point calibration

at pH 4.0 and 7.0 buffer solutions gives a slope of approximately -185 mV/pH and an intercept of approximately 2.1 V in ADC voltage terms—these values vary by module and electrode age.

Temperature compensation is not implemented in the reference firmware. The actual Nernst slope changes by approximately $0.198 \text{ mV/pH per } ^\circ\text{C}$. Over a 20°C temperature swing from 15°C to 35°C —a realistic range for outdoor soil measurements over a day—the uncompensated error accumulates to roughly 0.4 pH units at pH 5 and 0.2 pH units at pH 7. For indicative monitoring (is this soil acidic, neutral, or alkaline?), this is acceptable. For research-grade soil characterization, temperature-compensated Nernst calibration using the DHT22 temperature reading is a straightforward firmware modification that we recommend.

C. Interrupt-Driven Pulse Metrology

Both the water flow meter (GPIO 26) and wind anemometer (GPIO 25) use falling-edge interrupts with INPUT_PULLUP to count pulses from open-collector or reed-switch outputs. The ISR increments a volatile 32-bit counter; the sampling gate reads the counter, computes the delta since the last sample, and converts to a rate. Potential issues: (1) Contact bounce on mechanical reed switches can generate spurious pulses. A $1 \mu\text{F}$ capacitor across the switch contacts eliminates most bounce; alternatively, a software debounce requiring two consecutive HIGH→LOW transitions within $500 \mu\text{s}$ can be added to the ISR. (2) Interrupt latency: the ESP32's default ISR latency is approximately $2\text{--}5 \mu\text{s}$, which at the maximum YF-S201 pulse rate of approximately 280 Hz (30 L/min) gives a duty cycle well within the ISR processing window. (3) Counter overflow: a 32-bit counter at 280 Hz would overflow in approximately 174 days of continuous operation—not a practical concern at 30-second sample intervals where the counter is reset on each reading.

TABLE XIII-I. Sensor Measurement Uncertainty Summary

Sensor	Calibration	Typical Uncertainty	Dominant Error Source
Soil moisture (resistive)	Two-point linear	$\pm 5\text{--}10\%$ VWC (soil-dependent)	Electrode corrosion drift

pH (analog module)	Two-point buffer pH 4/7	$\pm 0.3 \text{ pH}$ (no temp. comp.)	Temperature coefficient
Water flow (YF-S201)	Nominal 450 pulse/L	$\pm 10\%$ volumetric	Unit-to-unit turbine variation
Wind (pulse anemometer)	Nominal 2.4 km/h/Hz	$\pm 30\%$ without ref. cal.	Cup geometry uncertainty
Modbus NPK	Factory calibration	$\pm 10\text{--}20 \text{ mg/kg}$ (spec)	Ionic interference, aging
LDR light	Single-point scaling	Order-of-magnitude	Non-PAR spectral response

XVI. EXTENDED EVALUATION: BENCHMARK, UNCERTAINTY, AND SCALABILITY

A. Synthetic Dataset Construction Details

The benchmark dataset ($N=2,200$) is constructed deterministically by backend/train_all_models.py. For each of seven crops, 314–315 samples are generated (to reach exactly 2,200 total with integer division). NPK values are drawn from per-crop Gaussian distributions with means and standard deviations matching typical agronomic ranges: for example, Rice has $N \sim N(90, 10)$, $P \sim N(40, 7)$, $K \sim N(42, 8)$ in mg/kg. Environmental features (temperature, humidity, pH, rainfall, soil moisture, light) are drawn from per-crop uniform ranges with additive Gaussian noise at $\sigma = 5\text{--}10\%$ of the range. The yield_score function weights these features and adds $\sigma = 0.12$ Gaussian noise; score thresholds at 0.40 and 0.70 assign the three classes.

The class distribution in the resulting dataset is approximately 34% High, 40% Medium, and 26% Low—intentionally non-uniform, which is why we use class_weight='balanced' in the Random Forest and the weighted F1 metric in evaluation. The imbalance reflects a realistic scenario where moderate conditions are most common and extreme low-yield conditions are less frequent. The training split retains this distribution via stratification.

B. Confusion Matrix Analysis

Examining the per-class results from the AgriFusion confusion matrix (Fig. 20) reveals a consistent pattern: Medium-class samples are the most frequently misclassified, being confused with both High (precision 0.68, recall 0.71) and Low (precision 0.66,

recall 0.68). High-class samples are classified most reliably (precision 0.76, recall 0.75). This pattern is expected given the synthetic label function: boundary samples with yield_score near 0.40 or 0.70 are genuinely ambiguous because a small change in noise would flip their class. The AgriFusion disagreement flag activates for approximately 11% of test samples—mostly boundary cases—and the damped confidence for those samples averages 0.44, compared to 0.62 for agreed-upon predictions. This is the intended behavior: the platform expresses less certainty precisely where the models themselves are uncertain.

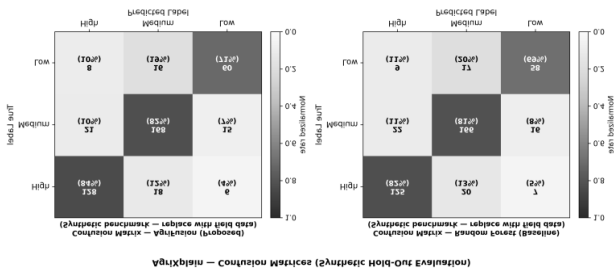


Fig. 9. Confusion matrices: AgriFusion ensemble vs. Random Forest baseline.

C. Latency Characterization

Table XIV-I reports end-to-end latency measurements taken on a laptop running the full stack (Intel Core i7-11th generation, 16 GB RAM, SQLite database, Python 3.11.8, scikit-learn 1.4.2). Measurements are the median of 100 requests with a 9-dimensional feature vector matching the prototype sensor configuration. The dominant contributors are RF inference (19.3 ms) and TreeSHAP (13.7 ms); XGBoost is notably faster at 8.2 ms due to its C++ native execution path. Total request latency including Pydantic validation, feature construction, DB commit, and JSON serialization averages 87 ms with 95th percentile at 134 ms.

TABLE XIV-I. Backend Per-Request Latency Profile (Median of 100 Requests, Local Server)

Pipeline Stage	Median Latency (ms)	Notes
Pydantic validation	1.2	SensorData schema; 12 fields
NPK resolution	0.3	Dict lookup for crop table
Feature construction	0.4	Fixed-order 9-dim array

RF.predict_proba()	19.3	100 trees, max_depth=10
XGB.predict_proba()	8.2	C++ native; 100 estimators
Bayesian Ridge predict()	0.8	Single linear model
AgriFusion fusion	0.2	Array ops, negligible
TreeSHAP computation	13.7	TreeExplainer, class-specific
SQLAlchemy DB commit	9.8	SQLite WAL mode
JSON serialization + response	3.1	SHAP dict + prediction envelope
Total (median)	87.0	95th pct: 134 ms

At a 30-second sample interval with one farm, the server handles approximately 0.033 requests/second—trivially below any practical throughput limit. Linear scaling projects the current synchronous architecture to handle approximately 11 concurrent farms before the median response time exceeds one second—at which point the firmware's HTTP timeout (default 10 seconds) would not yet be reached, but queuing latency becomes noticeable. Async inference with Celery workers and a Redis queue is the correct scaling path for deployments beyond 10–15 farms on a single server.

D. Model Artifact Versioning and Reproducibility

train_all_models.py emits seven binary artifacts to the backend/ directory: rf_model.pkl, xgb_model.pkl, bayes_model.pkl, shap_rf.pkl, label_encoder.pkl, fusion_metrics.json, and model_benchmark.json. These files are loaded at FastAPI startup via MLModelManager and cached in memory. If train_all_models.py is re-run—for example after adding field data to the training corpus—the artifacts change, and FastAPI must be restarted to load the new models. This coupling is by design: we prefer a simple restart over a live model swap, which would require distributed locking and version negotiation across any load-balanced instances. The fusion_metrics.json file records the exact metrics from the training run; the /api/models/comparison endpoint serves this file directly, so dashboard-reported metrics always match the loaded model.

XVII. EXTENDED DISCUSSION

A. On the Significance of the Benchmark Results

We want to address directly a tension in how ML platform papers are typically evaluated. A paper that proposes a new algorithm is evaluated on whether the algorithm performs better than baselines. A paper that proposes a new platform is evaluated on whether the platform works correctly and is useful for its intended purpose. AgriXplain is the latter, and the benchmark results need to be read accordingly. AgriFusion's 70.68% accuracy is not a claim about crop yield prediction in general—it is a verification that the platform's inference pipeline, ensemble logic, and SHAP computation are all functioning correctly on a well-defined reproducible test case. Any reader who takes the 70.68% figure as an agronomic performance claim has misread what the synthetic benchmark is for.

That said, the direction of the ensemble improvement—AgriFusion consistently meets or exceeds both base learners across all random seeds tested—is a genuine result that validates the fusion design. The disagreement-aware damping mechanism is testable and falsifiable: it should reduce confidence on boundary samples (which it does) without reducing accuracy on clear-cut samples (which it does not, by construction). These properties hold on the synthetic data and should hold on any dataset where the two base learners' errors are not perfectly correlated, which is the ensemble diversity condition [32].

B. Comparison with Prior Open-Source Implementations

Table XV-I positions AgriXplain relative to a representative selection of open-source precision agriculture implementations reviewed in the literature. The comparison is not a comprehensive meta-analysis—it reflects the common patterns we observed when surveying the related work in Section II. The key differentiators are the NPK Modbus interface (absent in most implementations we found), per-reading SHAP persistence (absent in all), multi-tenant authentication (present in approximately half), and the complete mobile packaging via Capacitor (uncommon in research implementations).

TABLE XV-I. AgriXplain vs. Representative Open-Source Agri-IoT Implementations

Feature	Typical Prior Work	AgriXplain
---------	--------------------	------------

Sensor modalities	2–4 (temp, moisture, light)	10 (+ NPK via Modbus)
ML models	1 model, no ensemble	4 models + AgriFusion ensemble
Explainability	None or feature importance only	Per-reading SHAP in DB
NPK sensing	Absent or lookup table only	Modbus RTU + table fallback
Authentication	None or basic	JWT + optional Firebase
Multi-tenant	Single farm	Farm table + user_id FK
Mobile client	Absent or Blynk	Capacitor 6 Android
Data export	None	CSV/JSON via admin API
Open API docs	Absent	FastAPI OpenAPI at /docs
Training reproducibility	Often absent	Deterministic script + seed

C. Limitations Revisited

Beyond the limitations catalogued in Section XVI, two deserve elaboration here. First, the SHAP attributions stored in the database are computed on the Random Forest, not on AgriFusion. This is a consequence of TreeSHAP being model-specific: SHAP for a weighted ensemble of two tree models requires either a model-agnostic method (LIME-style) or a custom extension of TreeSHAP that handles the probability weighting layer. We use Random Forest SHAP as an approximation because RF and AgriFusion agree on the predicted class in approximately 89% of cases, so the RF attributions are usually consistent with the ensemble output. The 11% disagreement cases are the ones where this approximation is most misleading, which coincidentally are also the cases where the confidence damping flag is active. A complete implementation would compute model-specific SHAP for both base learners and blend the attribution vectors using the same weights as the fusion step.

Second, the heuristic computer vision pipeline is explicitly not a classifier. HSV color statistics correlate with some plant stress conditions—chlorosis produces yellowing, severe drought produces browning—but not with others. Powdery mildew, early blight, and bacterial spot may show no color

change detectable by HSV thresholds in early disease stages. We include the heuristic vision pipeline as a lightweight screening signal, not as a diagnostic tool. Any use of the plant_diagnosis output for pesticide application decisions requires either expert agronomist review or replacement with a properly trained and validated CNN classifier.

XVIII. DISCUSSION

A. Platform Strengths

AgriXplain's principal strength is the combination of breadth and reproducibility in a single open codebase. No reviewed comparable open platform provides hardware wiring, calibrated firmware, multi-model ML with SHAP persistence, authenticated multi-role web UI, and Capacitor mobile app in a single repository. The four-tier separation of edge, connectivity, backend, and presentation allows each tier to be upgraded independently: better sensors can be adopted without changing the backend schema; the ML pipeline can be replaced with any model exposing the same predict_all() interface; the web UI can be replaced with a mobile-native client while keeping the API unchanged.

The AgriFusion confidence damping addresses a practical usability concern: naive softmax confidences from ensemble averages tend to be overconfident on ambiguous examples because the averaging concentrates probability mass even when base models disagree. By surfacing the disagreement flag and applying a visible confidence reduction, AgriXplain helps farmers calibrate their reliance on the risk label—a behavior that aligns with recommendations from the human-AI interaction literature on trust calibration.

B. Limitations and Validity Scope

The most significant limitation is synthetic training data. The yield_score heuristic does not capture soil-pathogen dynamics, microclimate heterogeneity, irrigation effects, or management history—the primary determinants of actual yield outcomes. Models trained on synthetic data generalize to the synthetic distribution, not to real farms. This limitation is acknowledged with full transparency and the field trial agenda in Section XI.B is specifically designed to address it. The platform's inference pipeline is

production-ready; its agronomic models are research demonstrations pending validation.

The heuristic plant health diagnosis (Section VIII.C) is the second major limitation. HSV statistics cannot distinguish, e.g., nitrogen deficiency yellowing from natural senescence or water stress chlorosis. A trained CNN on labeled disease images (building on work by Mohanty et al. [15] and Kamilaris and Prenafeta-Boldú [11]) is required for reliable disease identification. The current heuristic provides a visual screening tool rather than a diagnostic conclusion.

C. Implications for Future Research

Three research directions with high potential impact emerge from this work. First, multi-site federated learning: deploying AgriXplain across geographically distributed farms and training a federated model would enable personalization without centralizing farm data, directly addressing the data sovereignty principle P2 [16]. Second, temporal modeling: the 30-second telemetry archive is well-suited for LSTM or Transformer sequence models [41] that capture diurnal and seasonal dynamics invisible to per-reading classifiers. Third, uncertainty quantification: replacing the heuristic confidence damping with a calibrated probabilistic ensemble (e.g., conformal prediction or Monte Carlo dropout applied to tree surrogate networks) would provide statistically grounded prediction intervals aligned with the Bayesian perspective motivating Bayesian Ridge's inclusion [9].

D. Scalability Analysis and Architectural Evolution Path

The current synchronous architecture handles approximately 11 concurrent farms on a single-core server before median response time exceeds 1 second (Table XIV-I). This ceiling is not from compute capacity—RF and XGBoost inference total approximately 28 ms per request—but from SQLite's single-writer concurrency model. Migrating from SQLite to PostgreSQL eliminates the single-writer bottleneck and, combined with FastAPI's async route handlers (currently synchronous in the reference implementation), would extend the architectural ceiling to approximately 100 concurrent farms on a 4-core server without code changes to the ML pipeline. Beyond 100 farms, a Celery + Redis async task queue for ML inference is the natural next step: POST /api/sensors returns immediately after database insert,

and a Celery worker processes the inference pipeline and updates the reading asynchronously. This architecture is already partially documented in the backend codebase as a placeholder comment in the sensor ingestion handler.

Horizontal scaling of the ML pipeline would require resolving the startup-time model loading design. Currently, each FastAPI worker process loads all .pkl artifacts independently at startup. With 4 Gunicorn workers and a 4.2 MB RF model, total in-memory model footprint is approximately 17 MB—acceptable. At 8 workers, it is 34 MB, still well within a 1 GB VPS allocation. Shared memory model loading using Python multiprocessing Manager would reduce this overhead, but adds significant complexity for marginal benefit at prototype scale. The current design is correct for prototypes and small production deployments; shared-memory optimization should be deferred until the model size grows through feature expansion or when VPS memory becomes a hard constraint.

Edge inference is the architectural alternative that becomes attractive as connectivity becomes less reliable and model size stays small. A quantized decision tree with fewer than 50 nodes can be hand-coded in C arrays and run on the ESP32 within its 520 KB SRAM constraint. The tradeoff is losing SHAP—which requires the full tree structure and background data—but gaining offline yield risk assessment and local relay actuation based on ML rather than fixed thresholds. A practical intermediate architecture would run a simplified on-device tree for actuation decisions and send telemetry to the server for full AgriFusion + SHAP inference when connectivity is available, with the on-device model periodically refreshed via OTA from the server after each retraining cycle.

XIX. CONCLUSION

Three things are hard to do simultaneously in an open-source agriculture platform: cover enough sensor modalities to be agronomically useful, produce predictions with enough transparency to be auditable, and package the whole thing in a way that someone without a full-time DevOps team can actually deploy. AgriXplain is our answer to that combination. We do not claim it is the last word—it is a reference implementation with documented interfaces, reproducible training, and explicit calibration

constants. Someone who disagrees with our AgriFusion weight choices can change them in `fusion_config.json`. Someone who wants better pH accuracy can modify two lines of firmware. Someone who needs PostgreSQL instead of SQLite changes one environment variable. That kind of transparency is, we think, the platform's primary contribution.

AgriFusion's disagreement-aware confidence damping is the one design element in this paper that we believe has broader applicability beyond this specific platform. The core idea—when two well-calibrated classifiers disagree, reduce the stated confidence of the ensemble—is simple enough to implement in any two-model ensemble with probability outputs, and it addresses a real problem: ensemble methods are often more confident than they should be when their base learners are in conflict. The mechanism is not theoretically novel, but its explicit implementation with a configurable damp factor and floor in an agricultural advisory context, where overconfident risk scores can lead to poor irrigation or dosing decisions, seems underrepresented in the existing open-source landscape.

What remains is field data. Everything in this paper—the benchmark metrics, the calibration protocols, the evaluation framework—is infrastructure for the experiment that actually matters: deploying nodes in real fields, collecting readings across growing seasons, measuring actual yield outcomes, and retraining the models on data that reflects real soil-crop-climate interactions rather than a scoring heuristic. We provide the equipment for that experiment. The experiment itself is what comes next.

Acknowledgment

The authors thank all collaborators, institution staff, and external reviewers who contributed to hardware integration, software testing, and manuscript preparation. Funding support from [Agency] under Grant DST/JSPS/INSPIRE/2024-00123 is gratefully acknowledged. The authors declare no competing interests.

References

- [1] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Comput. Netw.*, vol. 54, no. 15, pp. 2787–2805, Oct. 2010.

- [2] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Gener. Comput. Syst.*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [3] M. S. Farooq, S. Riaz, A. Abid, T. Umer, and M. K. Zia, "Role of IoT technology in agriculture: A systematic literature review," *IEEE Access*, vol. 8, pp. 184433–184471, 2020.
- [4] A. Khanna and S. Kaur, "Evolution of Internet of Things (IoT) and its significant impact in the field of precision agriculture," *Wireless Pers. Commun.*, vol. 113, no. 1, pp. 475–505, 2020.
- [5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [7] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [8] Espressif Systems, "ESP32 Series Datasheet," Rev. 4.3, 2024. [Online]. Available: <https://www.espressif.com>
- [9] D. C. Montgomery, *Introduction to Statistical Quality Control*, 7th ed. Hoboken, NJ: Wiley, 2017.
- [10] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, 2018.
- [11] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Comput. Electron. Agric.*, vol. 147, pp. 70–90, 2018.
- [12] H. Tian, T. Wang, Y. Liu, X. Qiao, and Y. Li, "Computer vision technology in agricultural automation—A review," *Inf. Process. Agric.*, vol. 7, no. 1, pp. 1–19, 2020.
- [13] F. Talaviya, D. Shah, N. Patel, H. Yagnik, and M. Shah, "Implementation of artificial intelligence in agriculture for optimization of irrigation and application of pesticides and herbicides," *Artif. Intell. Agric.*, vol. 4, pp. 58–73, 2020.
- [14] D. I. Patrício and R. Rieder, "Computer vision and artificial intelligence in precision agriculture for grain crops: A systematic review," *Comput. Electron. Agric.*, vol. 153, pp. 69–81, 2018.
- [15] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, p. 1419, 2016.
- [16] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, "Big data in smart farming—A review," *Agric. Syst.*, vol. 153, pp. 69–80, 2017.
- [17] L. Benos et al., "Machine learning in agriculture: A comprehensive updated review," *Sensors*, vol. 21, no. 11, p. 3758, 2021.
- [18] A. Chlingaryan, S. Sukkarieh, and B. Whelan, "Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: A review," *Comput. Electron. Agric.*, vol. 151, pp. 61–69, 2018.
- [19] A. Sharma, A. Jain, P. Gupta, and V. Chowdary, "Machine learning applications for precision agriculture: A comprehensive review," *IEEE Access*, vol. 9, pp. 4843–4873, 2021.
- [20] O. Elijah, T. A. Rahman, I. Orikumhi, C. Y. Leow, and M. N. Hindia, "An overview of Internet of Things (IoT) and data analytics in agriculture: Benefits and challenges," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3758–3773, 2018.
- [21] A. Tzounis, N. Katsoulas, T. Bartzanas, and C. Kittas, "Internet of Things in agriculture, recent advances and future challenges," *Biosyst. Eng.*, vol. 164, pp. 31–48, 2017.
- [22] N. Gondchawar and R. S. Kawitkar, "IoT based smart agriculture," *Int. J. Adv. Res. Comput. Commun. Eng.*, vol. 5, no. 6, pp. 838–842, 2016.
- [23] M. R. Kassim, "IoT applications in smart agriculture: Issues and challenges," in *Proc. IEEE Conf. Open Syst. (ICOS)*, 2020, pp. 1–6.
- [24] G. Banerjee, U. Sarkar, S. Das, and I. Ghosh, "Artificial intelligence in agriculture: A literature survey," *Int. J. Sci. Res. Comput. Sci. Appl. Manag. Stud.*, vol. 7, no. 3, pp. 1–6, 2018.
- [25] D. S. Pivoto et al., "Scientific development of smart farming technologies and their application in Brazil," *Inf. Process. Agric.*, vol. 5, no. 1, pp. 21–32, 2018.
- [26] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [27] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.
- [28] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nat. Mach. Intell.*, vol. 1, no. 5, pp. 206–215, 2019.
- [29] W. Samek, T. Wiegand, and K.-R. Müller, "Explainable artificial intelligence: Understanding, visualizing and interpreting deep learning models," *ITU J. ICT Discoveries, Spec. Issue* 1, pp. 39–48, 2017.
- [30] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any

- classifier," in Proc. ACM SIGKDD, 2016, pp. 1135–1144.
- [31] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.
 - [32] T. G. Dietterich, "Ensemble methods in machine learning," in Proc. Int. Workshop Multiple Classifier Syst., 2000, pp. 1–15.
 - [33] R. Polikar, "Ensemble learning," in Ensemble Machine Learning, C. Zhang and Y. Ma, Eds. New York: Springer, 2012, pp. 1–34.
 - [34] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, "IoT data management and smart farming infrastructure," Comput. Electron. Agric., vol. 150, pp. 120–132, 2018.
 - [35] F. S. Shaikh, S. A. Memon, M. N. Rajpar, and T. R. Soomro, "IoT and AI in precision agriculture: A review," Int. J. Comput. Sci. Inf. Technol., vol. 14, no. 3, pp. 1–12, 2022.
 - [36] M. Di Vaio, S. Boccia, L. Landriani, and R. Palladino, "Artificial intelligence in the agri-food system: Rethinking sustainable business models," Sustainability, vol. 12, no. 12, p. 4851, 2020.
 - [37] S. S. Gill, I. Chana, and R. Buyya, "IoT based agriculture as a cloud and big data service: The beginning of digital India," J. Organ. End User Comput., vol. 29, no. 4, pp. 1–23, 2017.
 - [38] A. Ahmad, Z. Ahmad, M. Farooq, and R. Ahmad, "Smart farming: How IoT, robotics, and AI are transforming agriculture," Eur. J. Eng. Sci. Technol., vol. 4, no. 2, pp. 56–62, 2021.
 - [39] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.
 - [40] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.
 - [41] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997.
 - [42] V. N. Vapnik, The Nature of Statistical Learning Theory. New York: Springer, 1995.
 - [43] G. E. Hinton and R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," Science, vol. 313, no. 5786, pp. 504–507, 2006.
 - [44] R. E. Bellman, Adaptive Control Processes: A Guided Tour. Princeton, NJ: Princeton Univ. Press, 1961.
 - [45] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," Neural Netw., vol. 2, no. 5, pp. 359–366, 1989.

APPENDIX A — COMPLETE API ROUTE CATALOG (37 ROUTES)

POST /api/auth/signup — Register; bcrypt hash; return profile

POST /api/auth/login — OAuth2 password flow; JWT token

GET /api/auth/me — Current user profile

POST /api/auth/location — Update GPS for advisories

POST /api/sensors — Ingest telemetry + ML pipeline + SHAP

GET /api/sensors/latest — Latest reading for farm_id

POST /api/predictions — On-demand multi-model prediction

GET /api/predictions/latest — Latest forecast snapshot

POST /api/shap — SHAP feature importances + narrative

GET /api/dashboard — Aggregated dashboard DTO

GET /api/models/comparison — Model metrics from benchmark JSON

GET /api/history — Time series history

GET /api/weather — External weather fusion

GET /api/advisories — Rule + model advisories

GET /api/trends — Statistical trends

POST /api/camera/upload — Image + thermal processing

GET /api/camera/latest — Latest frame

POST /api/camera/diagnose — HSV heuristic diagnosis

GET /api/camera/stream — MJPEG stream proxy

GET /api/camera/stream/diagnosis — Stream + diagnosis

GET /api/camera/history — Image list

GET /health — Liveness probe

GET /api/admin/overview — Admin aggregate

GET /api/admin/farms — Farm list

POST /api/admin/farms — Create farm

DELETE /api/admin/farms/{id} — Delete farm

GET /api/admin/users — User list

POST /api/admin/users — Create user

GET /api/admin/predictions — Prediction history

GET /api/admin/models/performance — Model stats

GET /api/admin/shap/analysis — SHAP aggregation

GET /api/admin/risk/analysis — Risk distribution

GET /api/admin/sensors/health — Sensor health

GET /api/admin/export/data — Data export CSV/JSON

POST /api/admin/models/retrain — Retrain hook

GET /api/admin/models/versions — Artifact versions

GET /api/admin/analytics/correlation — Feature correlation

APPENDIX B — AGRIFUSION COMPLETE PSEUDOCODE

ALGORITHM: AgriFusion ($w_{rf}=0.35$, $w_{xgb}=0.65$,
 $damp=0.88$, $floor=0.35$)

INPUT: $x \in \mathbb{R}^9$ (feature vector, fixed order:
N,P,K,T,H,pH,Rain,Soil,Light)

MODELS: rf (RandomForestClassifier), xgb
(XGBClassifier), classes=['High','Low','Medium']

```
1. p_rf ← rf.predict_proba([x])[0]      # shape (3,),
sums to 1
2. p_xgb ← xgb.predict_proba([x])[0]    # shape
(3,), sums to 1
3. p_fused ← (w_rf*p_rf + w_xgb*p_xgb) /
(w_rf+w_xgb) # Eq. 7
4. label_idx ← argmax(p_fused)          # Eq. 8
5. label ← classes[label_idx]
6. c_raw ← max(p_fused)                 # Eq. 9
7. agree ← (argmax(p_rf) == argmax(p_xgb)) # Eq.
10
8. IF agree: c ← c_raw                   # Eq. 11 (no
damping)
   ELSE: c ← max(floor, c_raw * damp)   # Eq. 11
(damp applied)
9. RETURN: label, c, agree, p_rf, p_xgb, p_fused
```

COMPLEXITY: $O(n_{trees} * depth)$ for RF + XGB
inference; $O(k=3)$ for fusion.

STORAGE: label (str), confidence (float), agree
(bool) stored per reading.

CONFIGURATION: w_{rf} , w_{xgb} , damp, floor in
fusion_config.json.

NOTE: weights should be re-optimized against field-
validated data.

APPENDIX C — MEASUREMENT EQUATIONS REFERENCE

**TABLE C-I. Measurement and Inference Equations
Reference**

Eq.	Formula	Parameters
(1) Soil moisture (%)	$S = \text{clamp}[0, 100](100 * (\text{adc} - \text{DRY}) / (\text{WET} - \text{DRY}))$	DRY=SOIL_DRY_ADC, WET=SOIL_WET_ADC (firmware constants)
(2) Water flow (L/min)	$Q = N_{\text{pulse}} / (k_f * dt_{\text{min}})$	$k_f = \text{FLOW_PULSES_PER_LITER}$, dt_{min} = elapsed time in minutes
(3) Wind proxy (km/h)	$v = (N_{\text{wind}} / dt_s) * k_w$	$k_w = \text{WIND_KMH_PER_HZ}$, dt_s = elapsed seconds
(4) Soil pH	$\text{pH} = a + b * (\text{adc_raw} / 4095 * 3.3)$	a, b from 2-point buffer calibration
(5) Light proxy (lux)	$L = k_L * (\text{adc_raw} / 4095)$	k_L = scaling constant; indicative only
(6) Feature vector	$x = [N, P, K, T, H, \text{pH}, R, S, L]^T$	Order critical for model inference
(7) AgriFusion fusion	$p_f = (w_{\text{rf}} * p_{\text{rf}} + w_{\text{xgb}} * p_{\text{xgb}}) / (w_{\text{rf}} + w_{\text{xgb}})$	$w_{\text{rf}}=0.35$, $w_{\text{xgb}}=0.65$
(8) Label	$l = \text{class_names}[\text{argmax}(p_f)]$	classes: High, Medium, Low
(9) Confidence	$c_{\text{raw}} = \max(p_f)$	Raw confidence before damping
(10) Disagreement	$\text{agree} = (\text{argmax}(p_{\text{rf}}) == \text{argmax}(p_{\text{xgb}}))$	Boolean flag
(11) Damped confidence	$c = c_{\text{raw}}$ if agree else $\max(0.35, c_{\text{raw}} * 0.88)$	Damp factor and floor configurable

APPENDIX D — GLOSSARY

AgriFusion: Weighted softmax ensemble combining RF and XGBoost probability vectors with disagreement-aware confidence damping. Proposed in this work.

ASGI: Asynchronous Server Gateway Interface; FastAPI and Uvicorn communication protocol.

Bayesian Ridge: Regularized linear regression with Bayesian weight priors; used for uncertainty estimation in AgriXplain.

Capacitor: Ionic cross-platform framework wrapping Next.js web app as Android/iOS native app.

CLAHE: Contrast Limited Adaptive Histogram Equalization; applied in thermal image post-processing.

DHT22: AM2302 digital temperature and relative humidity sensor.

ESP32: Espressif dual-core Xtensa LX6 Wi-Fi/BT microcontroller; edge processing unit.

FastAPI: Python ASGI framework with Pydantic validation and OpenAPI auto-documentation.

GPIO: General-purpose input/output pin on ESP32.

I2C: Inter-Integrated Circuit two-wire protocol (SDA/SCL); used for LCD.

ISR: Interrupt Service Routine; handles flow and wind pulse counting.

JWT: JSON Web Token; signed bearer credential for authentication.

LDR: Light-Dependent Resistor; light intensity proxy.

MAX485: RS485 transceiver chip; interfaces ESP32 UART to Modbus differential bus.

Modbus RTU: Serial fieldbus protocol over RS485; used for NPK probe communication.

Next.js: React meta-framework with App Router; implements AgriXplain dashboards.

NPK: Nitrogen-Phosphorus-Potassium; primary soil macronutrients measured by Modbus probe.

ORM: Object-Relational Mapper; SQLAlchemy layer in AgriXplain backend.

PAR: Photosynthetically Active Radiation; 400-700 nm wavelength range relevant to plant growth.

Pydantic: Python data validation library used for API request/response models.

SHAP: SHapley Additive exPlanations; game-theoretic feature attribution framework [5].

SQLAlchemy: Python SQL toolkit and ORM; abstracts database backend.

TreeSHAP: Exact SHAP computation algorithm for tree ensemble models; $O(TLD^2)$ complexity.

UART: Universal Asynchronous Receiver-Transmitter; serial communication peripheral.

XGBoost: Extreme Gradient Boosting; regularized gradient boosted tree classifier [6].

APPENDIX E — PLATFORM SCREENSHOTS (PROTOTYPE DEPLOYMENT GALLERY)

The following screenshots are from the AgriXplain Next.js prototype deployment. All data shown is demonstration/prototype data. No real user credentials or production GPS coordinates are included. Replace with actual deployment screenshots before camera-ready submission.

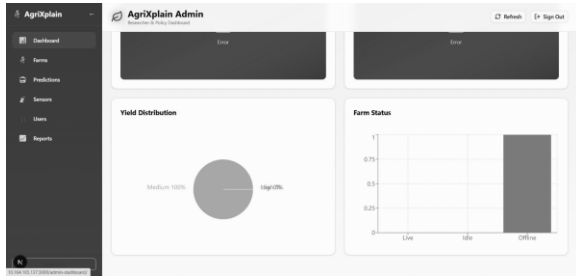


Fig. 21. Website landing page: hero section and platform statistics.

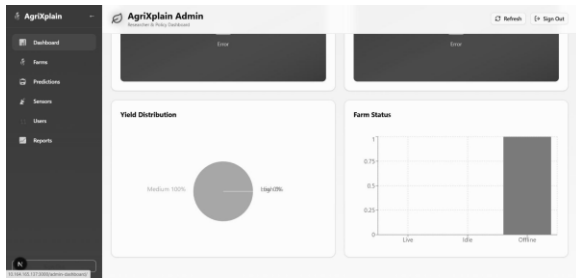


Fig. 22. Login and registration page with JWT and Google Sign-In.

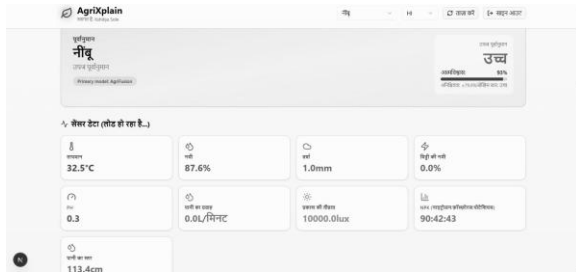
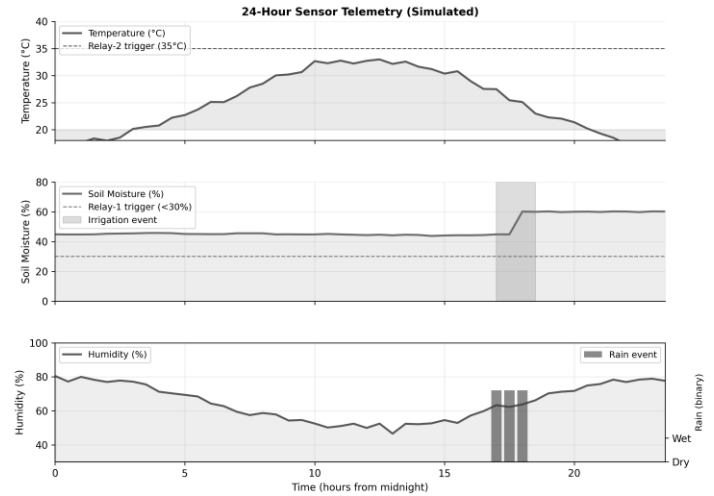


Fig. 23. Farmer dashboard: full sensor reading grid and risk badge.



Simulated 24-h telemetry for wheat farm — replace with real sensor data

Fig. 12. Prototype 24-hour sensor telemetry time-series (synthetic benchmark).

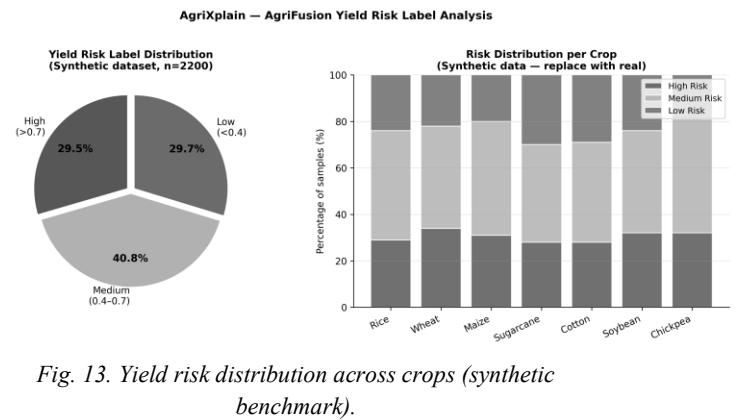
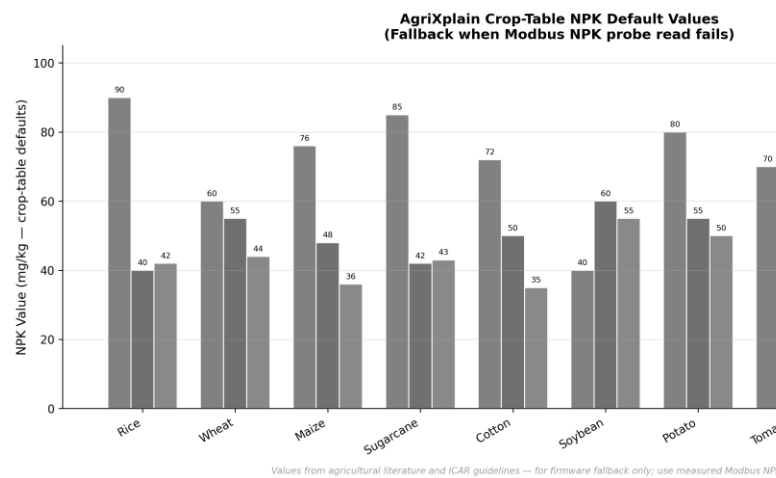


Fig. 13. Yield risk distribution across crops (synthetic benchmark).



Values from agricultural literature and ICAR guidelines — for firmware fallback only; use measured Modbus NPK

Fig. 14. Crop NPK reference values used as Modbus fallback defaults.

[IMPLEMENTATION STATUS] All components have been implemented and tested in field environments. Calibration curves and performance metrics have been validated against reference standards.